

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



TRẦN MẠNH DUY

**XÂY DỰNG ĐƯỜNG CONG ELLIPTIC BẬC SIÊU CAO VÀ ỨNG
DỤNG CHỮ KÝ SỐ
KHÓA LUẬN TỐT NGHIỆP**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY
Ngành: Công nghệ thông tin định hướng thị trường Nhật Bản

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

TRẦN MẠNH DUY

XÂY DỰNG ĐƯỜNG CONG ELLIPTIC BẬC SIÊU CAO VÀ ỨNG
DỤNG CHỮ KÝ SỐ
KHÓA LUẬN TỐT NGHIỆP

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY
Ngành: Công nghệ thông tin định hướng thị trường Nhật Bản

Cán bộ hướng dẫn: GS. TS. Lê Phê Đô

HÀ NỘI - 2026

LỜI CAM ĐOAN

Tôi xin cam đoan khóa luận tốt nghiệp *Xây dựng đường cong Elliptic bậc siêu cao và ứng dụng Chữ ký số* là do tôi tự tìm hiểu, nghiên cứu và phát triển dưới sự hướng dẫn của GS. TS. Lê Phê Đô, không sao chép hay sử dụng bất kỳ phần nào từ các công trình nghiên cứu của người khác mà không trích dẫn nguồn theo quy định. Tất cả những tài liệu tham khảo đều đã được liệt kê ở phần Tài liệu tham khảo của khóa luận. Tôi xin hoàn toàn chịu trách nhiệm về tính trung thực và chính xác của toàn bộ nội dung trong khóa luận này.

Tác giả khóa luận

Trần Mạnh Duy

LỜI CẢM ƠN

Viết lời cảm ơn ở đây...

MỤC LỤC

Lời cam đoan	i
Lời cảm ơn	v
Mục lục	ix
Tóm tắt	v
Danh mục viết tắt	vi
Danh mục mã nguồn	ix
Danh mục bảng biểu	x
Mở đầu	1
0.1. Đóng góp của đề tài	1
0.2. Bố cục của khóa luận	1
Chương 1. Tổng quan và Giới thiệu bài toán	3
1.1. Hệ mật mã khóa công khai và Đường cong Elliptic	3
1.2. Thực trạng các tiêu chuẩn ECC và Vấn đề tự chủ mật mã	3
1.3. Giới hạn bảo mật và nhu cầu đường cong bậc siêu cao	4
1.4. Vai trò của ngôn ngữ lập trình trong an toàn mật mã	4
1.5. Phát biểu bài toán và Phạm vi nghiên cứu	5
Chương 2. Cơ sở toán học	6
2.1. Định nghĩa Trường số nguyên tố	6
2.1.1. Cài đặt: <code>PrimeFieldConfig</code> và <code>PrimeFieldElement</code>	6
2.1.2. Các phép toán cơ bản	7
2.2. Đường cong elliptic Short Weierstrass	7
2.2.1. Định nghĩa	7
2.2.2. Cấu trúc nhóm	8
2.3. Phép cộng điểm (Point Addition)	8
2.3.1. Quy tắc hình học	8
2.3.2. Công thức đại số	8
2.3.3. Cài đặt	9
2.4. Phép nhân vô hướng (Scalar Multiplication)	9
2.4.1. Thuật toán Double-and-Add	9

2.4.2. Cài đặt	9
2.4.3. Tầm quan trọng	10
2.5. Tối ưu số học: Phép nhân Montgomery	10
2.5.1. Vấn đề	10
2.5.2. Biến đổi Montgomery	10
2.5.3. Chuyển đổi vào/ra không gian Montgomery	11
2.6. Lũy thừa nhanh (Square-and-Multiply)	11
Chương 3. Phương pháp xây dựng đường cong elliptic	13
3.1. Ý tưởng cốt lõi	13
3.1.1. Điều kiện CM và phương trình Diophantine	13
3.1.2. Đa thức lớp Hilbert $H_D(x)$	14
3.1.3. Quy trình CM đầy đủ	14
3.1.4. Trường hợp $D = -3$: Xoắn sextic	15
3.2. Tham số đầu vào của hệ thống	15
3.2.1. $k = 18$ — Bậc nhúng (Embedding Degree)	15
3.2.2. $d = 3$ — Biệt thức CM ($ D $)	15
3.2.3. <code>target_r_bits = 512</code> — Kích thước bậc nhóm r	16
3.2.4. <code>target_p_bits = 1024</code> — Kích thước trường cơ sở p	16
3.2.5. <code>max_attempts = 100_000</code> — Số lần thử tối đa	16
3.2.6. <code>min_scalar_two_adicity = 32</code> — Two-adicity tối thiểu của $r - 1$	16
3.2.7. <code>min_base_two_adicity = 32</code> — Two-adicity tối thiểu của $p - 1$	17
3.2.8. Tổng hợp: Đặc trưng đường cong mục tiêu	17
3.3. Thuật toán Cocks-Pinch truyền thống	18
3.3.1. Vấn đề: Xây dựng ngược từ r	18
3.3.2. Điều kiện bậc nhúng	18
3.3.3. Thuật toán CP cho $k = 18$	18
3.3.4. Phân tích độ phức tạp trung bình	19
3.4. Thuật toán Cocks-Pinch Cải tiến	19
3.4.1. Động lực: Tấn công rây trường số thấp (TNFS)	19
3.4.2. Yêu cầu NTT-friendly	20
3.4.3. Cải tiến 1: NTT-friendly r qua ràng buộc T	20

3.4.4.	Cải tiến 2: NTT-friendly p qua mở rộng lưới nâng	21
3.4.5.	Phân tích hiệu năng của thuật toán cải tiến	21
3.4.6.	So sánh với BLS12-381	22
3.4.7.	Giới hạn và hướng mở rộng	22
Chương 4.	Sơ đồ chữ ký số	24
4.1.	Cặp khóa (KeyPair)	24
4.1.1.	Hàm băm mở rộng	24
4.2.	Chữ ký Schnorr	25
4.2.1.	Lý thuyết	25
4.2.2.	Cấu trúc dữ liệu	25
4.2.3.	Cài đặt	26
4.2.4.	Bảo mật	26
4.3.	Chữ ký ECDSA	26
4.3.1.	Lý thuyết	26
4.3.2.	Cấu trúc dữ liệu	27
4.3.3.	Cài đặt	27
4.3.4.	So sánh Schnorr và ECDSA	27
4.4.	Chữ ký tổng hợp BLS	28
4.4.1.	Lý thuyết (tổng quan)	28
4.4.2.	Vai trò của đường cong KSS18	28
4.5.	Cài đặt hệ thống ký (curve1024-sig)	29
Chương 5.	Cài đặt và Đánh giá	30
5.1.	Đánh giá hiệu năng	30
5.1.1.	Kết quả đo đạc	30
5.1.2.	Phân tích: Kết quả có hợp lý về mặt lý thuyết không?	30
5.1.3.	Tính khả thi trong ứng dụng thực tế	31
5.2.	Đánh giá độ an toàn	31
5.2.1.	An toàn trước tấn công cổ điển	31
5.2.2.	An toàn trước tấn công Invalid Curve	32
5.2.3.	An toàn trước tấn công MOV	32
5.2.4.	An toàn trước tấn công Anomalous	33
5.2.5.	Hạn chế: Timing Side-Channel Attack	33

Chương 6. Kết luận	34
6.1. Kết quả đạt được	34
6.2. Đánh giá tổng quan	34
6.2.1. So sánh với mục tiêu ban đầu	34
6.2.2. Đánh giá điểm mạnh và hạn chế	35
6.3. Hướng phát triển	35
6.3.1. Tối ưu hóa hiệu năng (ngắn hạn)	35
6.3.2. Bảo mật cấp cao (trung hạn)	36
6.3.3. Mở rộng ứng dụng (dài hạn)	36
TÀI LIỆU THAM KHẢO	37

TÓM TẮT

Khóa luận trình bày quá trình thiết kế và cài đặt từ đầu (from scratch) một hệ thống mật mã đường cong elliptic (ECC) hoàn chỉnh bằng ngôn ngữ Rust, không phụ thuộc vào bất kỳ thư viện mật mã bên ngoài nào. Công trình tập trung vào hai đóng góp chính: (i) xây dựng lớp số học bignum an toàn bộ nhớ với phép nhân Montgomery và các phép toán số học cơ bản trên trường nguyên tố 1024-bit; (ii) chủ động sinh tham số đường cong pairing-friendly KSS18 với bậc nhúng $k = 18$ và bậc nhóm $r \approx 2^{512}$ thông qua thuật toán Cocks-Pinch cải tiến, đảm bảo tính kháng tấn công MOV, Anomalous và TNFS. Trên nền tảng này, khóa luận cài đặt thành công hai sơ đồ chữ ký số chuẩn mực là Schnorr và ECDSA, đồng thời xây dựng bộ kiểm thử toàn diện với 36/36 test cases PASSED, chứng minh tính đúng đắn của toàn bộ hệ thống. Kết quả cho thấy khả năng tự chủ hoàn toàn trong việc phát triển hạ tầng mật mã an toàn, minh bạch và có thể kiểm chứng, đặt nền móng cho các ứng dụng mật mã thế hệ mới.

Từ khóa: mật mã đường cong elliptic, Rust, số học bignum, phép nhân Montgomery, đường cong pairing-friendly, Cocks-Pinch, chữ ký số Schnorr, ECDSA, bảo mật 256-bit

DANH MỤC VIẾT TẮT

Viết tắt	Tên đầy đủ
AES	Advanced Encryption Standard — Tiêu chuẩn mã hóa nâng cao
AVX	Advanced Vector Extensions — Tập lệnh vector mở rộng (Intel/AMD)
BLS	Boneh-Lynn-Shacham — Sơ đồ chữ ký tổng hợp dựa trên ghép cặp
BN	Barreto-Naehrig — Họ đường cong elliptic pairing-friendly
CM	Complex Multiplication — Phương pháp Nhân phức
CP	Cocks-Pinch — Thuật toán sinh đường cong elliptic
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator — Bộ sinh số giả ngẫu nhiên an toàn mật mã
CVE	Common Vulnerabilities and Exposures — Danh mục lỗ hổng bảo mật
DL	Discrete Logarithm — Logarit rời rạc
DLP	Discrete Logarithm Problem — Bài toán logarit rời rạc
ECC	Elliptic Curve Cryptography — Mật mã đường cong Elliptic
ECDLP	Elliptic Curve Discrete Logarithm Problem — Bài toán logarit rời rạc trên đường cong Elliptic
ECDSA	Elliptic Curve Digital Signature Algorithm — Thuật toán chữ ký số đường cong Elliptic

Viết tắt	Tên đầy đủ
EUFCMA	Existential Unforgeability under Chosen Message Attack — Tính không thể giả mạo dưới tấn công thông điệp chọn trước
FFT	Fast Fourier Transform — Biến đổi Fourier nhanh
FIPS	Federal Information Processing Standards — Tiêu chuẩn xử lý thông tin liên bang (Hoa Kỳ)
FRI	Fast Reed-Solomon Interactive Oracle Proof — Chứng minh oracle tương tác Reed-Solomon nhanh
KSS	Kachisa-Schaefer-Scott — Họ đường cong elliptic pairing-friendly
KZG	Kate-Zaverucha-Goldberg — Sơ đồ cam kết đa thức
MOV	Menezes-Okamoto-Vanstone — Tấn công dùng phép ghép cặp
MSM	Multi-Scalar Multiplication — Phép nhân đa vô hướng
NAF	Non-Adjacent Form — Dạng biểu diễn không liền kề
NFS	Number Field Sieve — Thuật toán rây trường số
NIST	National Institute of Standards and Technology — Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ
NTT	Number Theoretic Transform — Biến đổi Lý thuyết số
PKC	Public Key Cryptography — Mật mã khóa công khai
PKI	Public Key Infrastructure — Hạ tầng khóa công khai

Viết tắt	Tên đầy đủ
PLONK	Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge — Hệ chứng minh ZK không tương tác
REDC	Montgomery Reduction — Phép rút gọn Montgomery
RSA	Rivest-Shamir-Adleman — Hệ mật mã khóa công khai RSA
SHA	Secure Hash Algorithm — Thuật toán băm an toàn
SIMD	Single Instruction, Multiple Data — Một lệnh, nhiều dữ liệu
SSSA	Smart-Satoh-Araki — Tấn công đường cong dị thường (Anomalous)
SSH	Secure Shell — Giao thức đăng nhập từ xa an toàn
TLS	Transport Layer Security — Bảo mật tầng giao vận
TNFS	Tower Number Field Sieve — Thuật toán rây trường số thấp
ZK	Zero-Knowledge — Chứng minh không tiết lộ thông tin

DANH MỤC MÃ NGUỒN

2.1	Định nghĩa PrimeFieldConfig và PrimeFieldElement	6
2.2	Cài đặt phép cộng trên trường hữu hạn modulo p	7
2.3	Cài đặt phép nghịch đảo bằng Định lý nhỏ Fermat	7
2.4	Định nghĩa cấu trúc điểm AffinePoint	8
2.5	Thuật toán nhân vô hướng Double-and-Add	10
2.6	Hàm giảm bậc Montgomery (REDC)	11
2.7	Phép gán hằng-thời-gian trong lũy thừa chống tấn công kênh kề	12
3.1	Cài đặt ràng buộc T để đạt two-adicity cho r	21
4.1	Cấu trúc dữ liệu cặp khóa KeyPair	24
4.2	Cấu trúc dữ liệu chữ ký Schnorr	25
4.3	Cấu trúc dữ liệu chữ ký ECDSA	27
4.4	Giao diện dòng lệnh CLI curve1024-sig	29

DANH MỤC BẢNG BIỂU

Bảng 2.1	Vai trò của phép nhân vô hướng trong các sơ đồ mật mã ECC	10
Bảng 2.2	So sánh chi phí phép nhân thông thường và Montgomery	11
Bảng 3.1	Đa thức lớp Hilbert $H_D(x)$ với các giá trị $ D $ nhỏ	14
Bảng 3.2	Tổng hợp đặc trưng đường cong mục tiêu	17
Bảng 3.3	So sánh thuật toán CP truyền thống và CP Cải tiến	22
Bảng 4.1	So sánh sơ đồ chữ ký Schnorr và ECDSA	27
Bảng 5.1	Kết quả đo hiệu năng hệ thống Curve1024	30
Bảng 6.1	Đối chiếu kết quả với mục tiêu đề ra	35

MỞ ĐẦU

Hệ mật mã Đường cong Elliptic (ECC) là nền tảng bảo mật của hạ tầng số hiện đại, từ giao thức TLS, SSH cho đến các hệ thống blockchain [1, 2]. Tuy nhiên, sự phụ thuộc vào các bộ tham số sinh sẵn bởi các tổ chức quốc tế đặt ra những thách thức nghiêm trọng về tự chủ công nghệ và an ninh quốc gia. Đồng thời, hầu hết các thư viện mật mã truyền thống được viết bằng C/C++ — một ngôn ngữ tiềm ẩn nhiều rủi ro về an toàn bộ nhớ.

Khóa luận này đặt mục tiêu thiết kế và cài đặt từ đầu (from scratch) một hệ thống mật mã đường cong elliptic trên trường nguyên tố cực lớn 1024-bit, sử dụng ngôn ngữ lập trình Rust. Hệ thống bao gồm toàn bộ ngăn xếp — từ lớp số nguyên lớn U_{1024} , số học trường hữu hạn Montgomery, phép toán nhóm trên đường cong, đến các sơ đồ chữ ký số Schnorr và ECDSA — không phụ thuộc bất kỳ thư viện mật mã bên ngoài nào.

0.1. Đóng góp của đề tài

Khóa luận đạt được các đóng góp chính sau:

1. **Xây dựng hệ thống số học bignum hoàn chỉnh** — Tự định nghĩa kiểu dữ liệu U_{1024} với phép nhân Montgomery, lũy thừa hằng-thời-gian, và nghịch đảo Fermat, đảm bảo an toàn bộ nhớ tuyệt đối nhờ kiến trúc Rust.
2. **Sinh đường cong elliptic mới bằng thuật toán Cocks-Pinch cải tiến** — Chủ động xây dựng đường cong pairing-friendly ($k = 18, r \approx 2^{512}, p \approx 2^{1024}$) với cấu trúc NTT-friendly trên cả hai trường, đạt mức bảo mật 256-bit và kháng tấn công TNFS [3, 4].
3. **Cài đặt ba sơ đồ chữ ký số** — Schnorr, ECDSA và chữ ký tổng hợp BLS, với bộ kiểm thử 36 test bao phủ từ số học cấp thấp đến mô phỏng tấn công MOV và Anomalous.

0.2. Bố cục của khóa luận

- **Chương 1. Tổng quan và Giới thiệu bài toán:** Bối cảnh lịch sử của ECC, thực trạng các tiêu chuẩn hiện hành, vấn đề tự chủ mật mã, vai trò của ngôn ngữ lập trình, và phát biểu bài toán.

- **Chương 2. Cơ sở toán học:** Trường hữu hạn, đường cong elliptic Short Weierstrass, phép cộng điểm, nhân vô hướng, và phép nhân Montgomery.
- **Chương 3. Phương pháp xây dựng đường cong elliptic:** Phương pháp Nhân phức (CM), thuật toán Cocks-Pinch truyền thống và cải tiến NTT-friendly.
- **Chương 4. Sơ đồ chữ ký số:** Cấu trúc cặp khóa, Schnorr, ECDSA, và chữ ký tổng hợp BLS.
- **Chương 5. Cài đặt và Đánh giá:** Benchmark hiệu năng, phân tích an toàn trước 5 loại tấn công, và định hướng phát triển.

CHƯƠNG 1

TỔNG QUAN VÀ GIỚI THIỆU BÀI TOÁN

Chương này cung cấp cái nhìn tổng quan về bối cảnh của Hệ mật mã khóa công khai, thực trạng của các tiêu chuẩn Đường cong Elliptic hiện hành, và phân tích những rủi ro bảo mật cốt lõi về cả mặt toán học lẫn triển khai phần mềm. Từ đó, chương làm nổi bật tính cấp thiết của bài toán xây dựng đường cong Elliptic bậc siêu cao bằng ngôn ngữ an toàn bộ nhớ.

1.1. Hệ mật mã khóa công khai và Đường cong Elliptic

Sự ra đời của Mật mã khóa công khai (Public Key Cryptography - PKC), đánh dấu bởi công trình của Diffie và Hellman năm 1976 [1], là một bước ngoặt trong lịch sử an toàn thông tin. Mô hình mã hóa sử dụng khóa bất đối xứng — một khóa công khai để mã hóa/xác minh và một khóa riêng tư để giải mã/ký số — đã giải quyết triệt để bài toán phân phối khóa qua kênh truyền không an toàn mà hệ mật mã khóa đối xứng truyền thống không thể làm được.

Tuy nhiên, các hệ mật mã khóa công khai thế hệ đầu như RSA đòi hỏi kích thước khóa rất lớn để đạt mức bảo mật cao. Để đạt mức 128-bit bảo mật đối xứng (mức tối thiểu được khuyến cáo hiện nay theo NIST SP 800-57 [5]), hệ thống RSA cần sử dụng khóa dài tới 3072-bit. Nhu cầu tối ưu hóa này dẫn tới sự ra đời của Hệ mật mã Đường cong Elliptic (Elliptic Curve Cryptography - ECC), được đề xuất độc lập bởi Koblitz [6] và Miller [7] vào giữa thập niên 1980. Sức mạnh vượt trội của ECC nằm ở tỷ lệ giữa kích thước khóa và độ an toàn: ECC chỉ yêu cầu khóa 256-bit để đạt mức bảo mật 128-bit tương đương RSA 3072-bit. Kích thước khóa nhỏ hơn đáng kể giúp ECC tiêu thụ ít tài nguyên tính toán, băng thông và điện năng hơn, biến nó trở thành tiêu chuẩn vàng cho các giao thức mạng hiện đại như TLS, mạng tiền điện tử (Bitcoin, Ethereum), và các hệ thống chữ ký số điện tử [2].

1.2. Thực trạng các tiêu chuẩn ECC và Vấn đề tự chủ mật mã

Mặc dù ECC mang lại lợi ích lớn về hiệu năng, việc triển khai ECC trong thực tế bộc lộ một vấn đề nghiêm trọng về chủ quyền công nghệ. Hiện nay, hầu hết các hệ thống phần mềm đều tái sử dụng các đường cong được định nghĩa và sinh sẵn bởi các tổ chức

quốc tế, điển hình như Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST, ví dụ tập tham số P-256) hoặc Tổ chức SECG (ví dụ secp256k1).

Việc sử dụng các hằng số toán học (các hệ số p, a, b) được sinh ra từ các “hạt giống” (seed) không giải thích được rõ ràng tiềm ẩn nguy cơ bị cài cắm “cửa hậu” (backdoor). Nguy cơ này không chỉ nằm trên lý thuyết mà đã trở thành hiện thực với sự kiện cơ quan tình báo NSA bị cáo buộc can thiệp và cài backdoor vào tiêu chuẩn thuật toán sinh số ngẫu nhiên Dual_EC_DRBG [8]. Sự kiện này đặt ra yêu cầu bức thiết về năng lực **“Tự chủ mật mã”** — nơi các hệ thống quốc gia có khả năng tự thiết kế, đánh giá và làm chủ các tham số đường cong bằng các phương pháp toán học minh bạch, không phụ thuộc vào các thư viện “hộp đen” từ nước ngoài.

1.3. Giới hạn bảo mật và nhu cầu đường cong bậc siêu cao

Xét về giới hạn an toàn trước máy tính cổ điển: khóa ECC 256-bit dù vẫn an toàn ở thời điểm hiện tại nhưng chỉ đạt biên độ an toàn 128-bit đối xứng. Thuật toán tấn công hiệu quả nhất là Pollard’s ρ [9] với độ phức tạp $O(\sqrt{r})$, trong đó r là bậc nhóm con nguyên tố.

Khóa luận này đặt mục tiêu đẩy cấp độ bảo mật lên bậc cực hạn: đường cong trên trường nguyên tố **1024-bit**. Với bậc nhóm $r \approx 2^{512}$, hệ thống cung cấp mức bảo mật xấp xỉ **256-bit đối xứng** — tương đương AES-256 và vượt xa chuẩn tối thiểu NIST. Tại mức này, thuật toán Pollard’s ρ cần $O(2^{256})$ phép toán nhóm, khiến mọi cuộc tấn công cổ điển trở thành bất khả thi tuyệt đối trong các giới hạn vật lý của vũ trụ.

Đây là sự đánh đổi có chủ ý: hiệu năng chậm hơn đáng kể so với đường cong 256-bit nhưng đổi lại biên an toàn vượt trội, phù hợp với kịch bản bảo mật dài hạn và các ứng dụng ký ngoại tuyến.

1.4. Vai trò của ngôn ngữ lập trình trong an toàn mật mã

Trong lịch sử ngành an toàn thông tin, những sự cố bảo mật nghiêm trọng nhất của các thư viện mật mã lớn thường không xuất phát từ sai sót trong toán học lý thuyết, mà bắt nguồn từ lỗi hỏng quản lý bộ nhớ của ngôn ngữ C/C++. Các lỗi tràn bộ đệm (buffer overflow), truy cập vùng nhớ đã giải phóng (use-after-free), hay đọc quá giới hạn vùng nhớ đã gây ra hàng loạt thảm họa bảo mật, nổi tiếng nhất là sự cố Heartbleed năm 2014 trong thư viện OpenSSL [10]. Theo thống kê từ Microsoft, khoảng 70% các bản vá lỗi bảo mật cốt lõi của họ có nguyên nhân trực tiếp từ nhóm lỗi an toàn bộ nhớ [11].

Thực tế triển khai hệ thống xử lý số nguyên lớn (BigInt) kích thước 1024-bit bằng

C/C++ là thách thức cực đoan về quản lý bộ nhớ. Khóa luận này khắc phục điểm yếu kiến trúc đó bằng ngôn ngữ lập trình hệ thống **Rust**. Với cơ chế Ownership (quyền sở hữu) và Borrow Checker (kiểm tra mượn tĩnh) tại thời điểm biên dịch, trình biên dịch Rust loại bỏ hoàn toàn các lỗi an toàn bộ nhớ mà không cần bộ thu gom rác (Garbage Collector). Cách tiếp cận này giúp thư viện mật mã đạt tốc độ thực thi ngang ngửa C/C++ trong khi được bảo hành tuyệt đối về an toàn bộ nhớ.

1.5. Phát biểu bài toán và Phạm vi nghiên cứu

Dựa trên những phân tích trên, khóa luận đặt ra mục tiêu thiết kế và cài đặt từ đầu (from scratch) một thư viện mã nguồn mở cho Hệ mật mã Đường cong Elliptic với các đặc điểm:

1. **Về mặt toán học:** Chủ động sinh mới đường cong Elliptic trên trường nguyên tố 1024-bit bằng phương pháp Nhân phức (CM) và thuật toán Cocks-Pinch cải tiến [3], tự chủ hoàn toàn các tham số mật mã.
2. **Về mặt kiến trúc phần mềm:** Mã nguồn phân tầng theo ba lớp biệt lập:
 - *Lớp Số nguyên lớn* (U1024): Kiểu dữ liệu 1024-bit với phép toán tối ưu phần cứng.
 - *Lớp Trường hữu hạn* (PrimeField): Số học modulo Montgomery bảo mật.
 - *Lớp Tọa độ điểm* (AffinePoint): Biểu diễn hình học trên đường cong.
3. **Về mặt ứng dụng:** Cài đặt, kiểm thử và đánh giá hiệu năng hai sơ đồ chữ ký số tiêu chuẩn (ECDSA [12] và Schnorr [13]) trên đường cong 1024-bit tự sinh.
4. **Về mặt công nghệ:** Toàn bộ hệ thống viết bằng Rust, đảm bảo an toàn bộ nhớ tuyệt đối và hiệu năng ở cấp ngôn ngữ hệ thống.

CHƯƠNG 2

CƠ SỞ TOÁN HỌC

Chương này trình bày các khái niệm toán học nền tảng được sử dụng trong toàn bộ luận văn: cấu trúc trường hữu hạn, đường cong elliptic Short Weierstrass và các phép toán nhóm trên nó [14, 15], cùng với các kỹ thuật số học tối ưu (Montgomery, Fermat) được cài đặt trực tiếp trong thư viện `curve1024`.

2.1. Định nghĩa Trường số nguyên tố

Trường số nguyên tố $\mathbb{F}_p$ là tập $\{0, 1, \dots, p-1\}$ với phép cộng và nhân lấy modulo p , trong đó p là số nguyên tố. Đây là trường hữu hạn nhỏ nhất có đặc số p .

Các tính chất quan trọng:

- **Đóng kín:** kết quả của mọi phép toán đều thuộc $\mathbb{F}_p$.
- **Nghịch đảo nhân:** mọi phần tử $a \neq 0$ có nghịch đảo a^{-1} sao cho $a \cdot a^{-1} \equiv 1 \pmod{p}$.
- **Theo Định lý nhỏ Fermat:** $a^{p-1} \equiv 1 \pmod{p}$ với mọi $a \neq 0$, do đó $a^{-1} = a^{p-2} \pmod{p}$.

2.1.1. Cài đặt: *PrimeFieldConfig* và *PrimeFieldElement*

Trong thư viện `curve1024`, trường $\mathbb{F}_p$ được biểu diễn qua trait `PrimeFieldConfig` (mang các hằng số tĩnh) và struct `PrimeFieldElement<C>` (phần tử trường):

```
pub trait PrimeFieldConfig {  
    const MODULUS: U1024;    // p: số nguyên tố trường  
    const R2: U1024;         // R^2 mod p, dùng cho Montgomery  
    const N_PRIME: U1024;    // -p^{-1} mod 2^{1024}, dùng cho REDC  
}  
  
pub struct PrimeFieldElement<C: PrimeFieldConfig> {  
    value: U1024,             // biểu diễn Montgomery: a.R mod p  
}
```

Mã nguồn 2.1. Định nghĩa PrimeFieldConfig và PrimeFieldElement

Thiết kế này cho phép trình biên dịch mở inline toàn bộ hằng số tại thời gian biên dịch (zero-cost abstraction): hai trường khác nhau $\mathbb{F}_p$ và $\mathbb{F}_r$ dùng chung code nhưng có tham số riêng biệt không thể nhầm lẫn.

2.1.2. Các phép toán cơ bản

Cộng và trừ thực hiện bằng cộng/trừ số nguyên rồi hiệu chỉnh modulo:

```
// Cộng: (a + b) mod p
fn add(self, rhs: Self) -> Self {
    let (sum, carry) = self.value.carrying_add(&rhs.value);
    let (sub, borrow) = sum.borrowing_sub(&C::MODULUS);
    // Chọn kết quả không cần phép chia
    conditional_select(sum, sub, carry || !borrow)
}
```

Mã nguồn 2.2. Cài đặt phép cộng trên trường hữu hạn modulo p

Phép cộng không cần phép chia đầy đủ — chỉ cần một phép trừ có điều kiện. Chi phí: $O(n)$ với n là số limb (16 limb x 64 bit = 1024 bit).

Nhân dùng phép nhân Montgomery (xem §2.4).

Nghịch đảo dùng lũy thừa Fermat: $a^{-1} = a^{p-2} \pmod{p}$, thực hiện bởi hàm `pow` với phép nhân bình phương liên tiếp (square-and-multiply):

```
pub fn inv(&self) -> Self {
    self.pow(C::MODULUS - 2) // a^(p-2) mod p
}
```

Mã nguồn 2.3. Cài đặt phép nghịch đảo bằng Định lý nhỏ Fermat

2.2. Đường cong elliptic Short Weierstrass

2.2.1. Định nghĩa

Trên $\mathbb{F}_p$ với $p > 3$, **đường cong elliptic dạng Short Weierstrass** là tập hợp:

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p \times \mathbb{F}_p \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

trong đó $\mathcal{O}$ là **điểm vô cực** (point at infinity) — phần tử đặc biệt không thuộc mặt phẳng affine, đóng vai trò phần tử đơn vị của nhóm.

Điều kiện không suy biến: Biệt thức $\Delta = -16(4a^3 + 27b^2) \neq 0$, đảm bảo đường cong không có điểm kỳ dị (cusp hay node). Kiểm tra này được thực hiện ngầm qua quá trình CM: với $D = -3$, luôn có $a = 0$ và điều kiện thu gọn thành $b \neq 0$.

2.2.2. Cấu trúc nhóm

Tập $E(\mathbb{F}_p)$ tạo thành một **nhóm Abel hữu hạn** với phép cộng điểm. Bậc của nhóm $\#E(\mathbb{F}_p) = p + 1 - t$ (định lý Hasse [14]), trong đó $|t| \leq 2\sqrt{p}$ là vết Frobenius.

Trong cài đặt:

```
pub struct AffinePoint<C: SWCurveConfig> {
    pub x: FieldElement<C::BaseField>,
    pub y: FieldElement<C::BaseField>,
    pub is_infinite: bool,
}
```

Mã nguồn 2.4. Định nghĩa cấu trúc điểm AffinePoint

2.3. Phép cộng điểm (Point Addition)

2.3.1. Quy tắc hình học

Phép cộng trên E được định nghĩa bằng quy tắc “dây cung và tiếp tuyến”:

- **Phần tử đơn vị:** $P + \mathcal{O} = \mathcal{O} + P = P$ với mọi P .
- **Phần tử nghịch đảo:** $-(x, y) = (x, -y)$, vì $P + (-P) = \mathcal{O}$.
- **Cộng hai điểm phân biệt** $P_1 \neq \pm P_2$: vẽ đường thẳng qua P_1, P_2 , điểm giao thứ ba với đường cong là R' , phản chiếu qua trục x cho $P_3 = P_1 + P_2 = -R'$.
- **Nhân đôi** $P + P = 2P$: dùng đường tiếp tuyến tại P .

2.3.2. Công thức đại số

Cho $P_1 = (x_1, y_1)$ và $P_2 = (x_2, y_2)$ trên $y^2 = x^3 + ax + b$, điểm tổng $P_3 = (x_3, y_3)$:

Cộng hai điểm phân biệt ($P_1 \neq P_2$):

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

Nhân đôi ($P_1 = P_2$):

$$\lambda = \frac{3x_1^2 + a}{2y_1}, \quad x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

Mỗi công thức cần **1 phép nghịch đảo** (tính λ), 2–3 phép nhân, và 4–6 phép cộng/trừ trong $\mathbb{F}_p$.

2.3.3. Cài đặt

Các bước cộng và nhân đôi điểm trên đường cong được cài đặt trực tiếp từ công thức đại số, thông qua hai hàm `add()` và `double()`. Đặc biệt, hệ thống tích hợp sẵn hàm `is_on_curve()` trong quá trình khởi tạo điểm kết quả thông qua hàm `new()` để kiểm tra và loại bỏ ngay lập tức các điểm không hợp lệ. Điều này giúp hệ thống bắt lỗi sớm và miễn nhiễm hoàn toàn với tấn công đường cong không hợp lệ (Invalid Curve Attack).

Chi tiết mã nguồn cài đặt các hàm `add()` và `double()` được trình bày tại mã nguồn công khai của dự án [16].

2.4. Phép nhân vô hướng (Scalar Multiplication)

2.4.1. Thuật toán Double-and-Add

Phép nhân vô hướng $[k]P = P + P + \dots + P$ (k lần) được tính hiệu quả bằng **Double-and-Add** (tương tự binary exponentiation), với độ phức tạp $O(\log k)$ thay vì $O(k)$:

Input: $P \in E(F_p)$, $k \in [0, r-1]$

Output: $[k]P$

```
R ← O (điểm vô cực)
B ← P
for i = 0 to 1023:
    if bit i của k bằng 1:
        R ← R + B
    B ← 2B
return R
```

Với $k \approx 2^{512}$ (1024 bit), thuật toán thực hiện tối đa 1024 lần nhân đôi và trung bình 512 lần cộng điểm.

2.4.2. Cài đặt

```
pub fn mul(&self, scalar: &U1024) -> Self {
    let mut result = Self::infinite();
    let mut base = *self;
    for i in 0..1024 {
        let limb_idx = i / 64;
        let bit_idx = i % 64;
        if (scalar.0[limb_idx] >> bit_idx) & 1 == 1 {
```

```

        result = result.add(&base);
    }
    base = base.double();
}
result
}

```

Mã nguồn 2.5. Thuật toán nhân vô hướng Double-and-Add

Mỗi lần gọi `mul` thực hiện đúng 1024 lần `double` và tối đa 1024 lần `add`. Chi phí tính toán chủ yếu đến từ phép nghịch đảo trong $\mathbb{F}_p$ (mỗi lần `add/double` cần 1 nghịch đảo, tức p^{p-2} — một lũy thừa 1024-bit).

2.4.3. Tầm quan trọng

Phép nhân vô hướng là hàm cốt lõi của mọi sơ đồ mật mã ECC:

Bảng 2.1. Vai trò của phép nhân vô hướng trong các sơ đồ mật mã ECC

Thao tác	Ký hiệu	Ứng dụng
Sinh khóa công khai	$Q = [d]G$	Từ khóa riêng d
Cam kết ký (Schnorr/ECDSA)	$R = [k]G$	Nonce commitment
Xác minh Schnorr	$[s]G$ và $[e]Q$	Hai lần nhân vô hướng
Xác minh ECDSA	$[u_1]G + [u_2]Q$	Hai lần nhân vô hướng

2.5. Tối ưu số học: Phép nhân Montgomery

2.5.1. Vấn đề

Phép nhân thông thường $a \cdot b \pmod{p}$ đòi hỏi một phép chia đầy đủ để lấy số dư — rất đắt với số 1024-bit. Montgomery đề xuất thực hiện phép nhân trong **không gian Montgomery** để tránh phép chia [17].

2.5.2. Biến đổi Montgomery

Chọn $R = 2^{1024}$ (cơ số nguyên tố cùng nhau với p). **Dạng Montgomery** của a là $\hat{a} = a \cdot R \pmod{p}$.

Phép nhân Montgomery (**REDC**) tính $\hat{a} \cdot \hat{b} \cdot R^{-1} \pmod{p}$ từ $\hat{a}$ và $\hat{b}$, chỉ dùng phép chia cho R (shift phải 1024 bit — rất rẻ) thay vì chia cho p .

Bảng 2.2. So sánh chi phí phép nhân thông thường và Montgomery

Phép tính	Thông thường	Montgomery
$a \cdot b \pmod{p}$	cần chia cho p	shift + trừ có điều kiện
Chi phí	$O(n^2)$ + chia	$O(n^2)$ không chia

Quy trình REDC (với ba hằng số biên dịch: $R^2 \pmod{p}$, $-p^{-1} \pmod{R}$):

1. Nhân $lo, hi = \hat{a} \cdot \hat{b}$
2. $m = lo \cdot N' \pmod{R}$ với $N' = -p^{-1} \pmod{R}$
3. $t = (hi, lo) + m \cdot p$ (chỉ giữ phần trên R)
4. Nếu $t \geq p$: $t \leftarrow t - p$

```
fn reduce(lo: &U1024, hi: &U1024) -> U1024 {
    let (m, _) = lo.widening_mul(&C::N_PRIME); // m = lo . N' mod R
    let (mn_lo, mn_hi) = m.widening_mul(&C::MODULUS); // m . p
    let (_, carry_lo) = lo.carrying_add(&mn_lo);
    let (t, carry_hi) = hi.carrying_add(&mn_hi);
    // Bước hiệu chỉnh cuối: trừ p nếu cần
    let (sub, borrow) = t.borrowing_sub(&C::MODULUS);
    if carry_hi || carry_lo { sub } else if !borrow { sub } else { t }
}
```

Mã nguồn 2.6. Hàm giảm bậc Montgomery (REDC)

2.5.3. Chuyển đổi vào/ra không gian Montgomery

- **Chuyển vào:** `PrimeFieldElement::new(a)` tính $a \cdot R^2 \cdot R^{-1} = a \cdot R \pmod{p}$.
- **Chuyển ra:** `to_u1024()` gọi `reduce(value, 0) = \hat{a} \cdot R^{-1} = a \pmod{p}`.

Trong thực tế, hầu hết tính toán diễn ra trong không gian Montgomery — chỉ chuyển ra khi cần so sánh hay xuất kết quả, giúp loại bỏ hoàn toàn phép chia trong vòng lặp tính toán chính.

2.6. Lũy thừa nhanh (Square-and-Multiply)

Hàm `pow` thực hiện lũy thừa $a^e \pmod{p}$ bằng phương pháp **square-and-multiply** (nhân bình phương liên tiếp). Việc cài đặt cấp thấp tiềm ẩn rủi ro lộ lọt thông tin khóa qua độ chênh lệch thời gian thực thi của các khối lệnh điều kiện.

Để bảo vệ hệ thống trước tấn công kênh kề (timing side-channel), thay vì sử dụng cấu trúc rẽ nhánh `if-else` (mở ra nguy cơ phán đoán bit số mũ dựa trên thời gian thực thi của CPU branch-prediction), thủ tục sử dụng cách tính chọn hằng-thời-gian (constant-time execution):

```
// Cả hai nhánh điều kiện đều được tính toán  
let product = res * base;  
// Chỉ định phép gán qua hàm chuyển đổi bit, không dùng rẽ nhánh điều khiển  
res = Self::conditional_select(&res, &product, bit.into());
```

Mã nguồn 2.7. Phép gán hằng-thời-gian trong lũy thừa chống tấn công kênh kề

Hàm `conditional_select` đảm bảo cả hai nhánh (khi bit bằng 0 hoặc 1) đều tiêu tốn một lượng chu kỳ lệnh (clock cycles) CPU bằng nhau hoàn toàn. Chi tiết mã nguồn vòng lặp 1024-bit của hàm `pow` được đính kèm tại mã nguồn công khai của dự án [16]. Nghịch đảo $a^{-1} = a^{p-2}$ dùng `pow` với $e = p - 2$: 1024 lần bình phương và ~512 lần nhân (trung bình), tổng ≈ 1536 phép nhân Montgomery.

CHƯƠNG 3

PHƯƠNG PHÁP XÂY DỰNG ĐƯỜNG CONG ELLIPTIC

Chương này trình bày trọng tâm lý thuyết của luận văn: các phương pháp xây dựng đường cong elliptic có bậc nhúng (embedding degree) xác định, phục vụ cho các ứng dụng mật mã dựa trên phép ghép cặp (pairing-based cryptography) [18]. Chúng ta sẽ đi từ nền tảng lý thuyết của Phương pháp Nhân phức (CM) và Đa thức lớp Hilbert [14], đến thuật toán Cocks-Pinch truyền thống [3], và cuối cùng là thuật toán Cocks-Pinch Cải tiến được đề xuất trong luận văn này, với khả năng sinh tham số trường cơ sở có cấu trúc NTT-friendly nhằm kháng lại tấn công rây trường số thấp (Tower Number Field Sieve - TNFS) [4].

3.1. Ý tưởng cốt lõi

Phương pháp Nhân phức (CM) là công cụ nền tảng để **xây dựng đường cong elliptic có số điểm định trước**. Thay vì chọn ngẫu nhiên các hệ số a, b rồi đếm điểm, CM đi ngược lại: bắt đầu từ bậc nhóm mong muốn $\#E(\mathbb{F}_p) = p + 1 - t$ rồi suy ngược ra phương trình đường cong.

Ý tưởng xuất phát từ lý thuyết số học phức: mỗi đường cong elliptic E sở hữu một **vành nội cấu** (endomorphism ring) $\text{End}(E)$. Với đường cong thông thường (không siêu dị), vành này đẳng cấu với một **thứ tự trong trường số** $\mathbb{Q}(\sqrt{D})$, trong đó $D < 0$ là số nguyên âm gọi là **biệt thức CM**. Giá trị D đặc trưng hoàn toàn cho “loại” đường cong về mặt số học phức.

3.1.1. Điều kiện CM và phương trình Diophantine

Để tồn tại một đường cong elliptic trên $\mathbb{F}_p$ có vết Frobenius t và biệt thức CM là D , điều kiện cần và đủ là phương trình Diophantine sau phải có nghiệm nguyên:

$$4p = t^2 + |D| \cdot v^2$$

trong đó p là đặc số trường, t là vết Frobenius, v là một số nguyên dương, và $D < 0$. Phương trình này **ràng buộc trực tiếp** bộ tham số (p, t, D) với nhau; không phải mọi bộ ba nào cũng có thể thỏa mãn đồng thời.

Ví dụ: Với $D = -3$ (biệt thức của secp256k1, BLS12-381), phương trình trở thành $4p = t^2 + 3v^2$.

3.1.2. Đa thức lớp Hilbert $H_D(x)$

Khi đã có D , bước tiếp theo là tính **Đa thức lớp Hilbert** (Hilbert class polynomial) $H_D(x)$. Đây là đa thức hệ số nguyên được xây dựng từ lý thuyết số học phức mà nghiệm phức của nó chính là các j -**invariant** của tất cả đường cong elliptic phức $\mathbb{C}/\mathcal{O}$ với thứ tự CM là $\mathcal{O}$.

Bậc của $H_D(x)$ bằng **số lớp** (class number) $h(D)$ — một bất biến số học của thứ tự $\mathbb{Z}[\sqrt{D}]$. Với $|D|$ nhỏ:

Bảng 3.1. Đa thức lớp Hilbert $H_D(x)$ với các giá trị $|D|$ nhỏ

$ D $	$h(D)$	$H_D(x)$
3	1	x
4	1	$x - 1728$
7	1	$x + 3375$
11	1	$x + 32768$
23	3	$x^3 + \dots$

Với $|D| = 3$ và $|D| = 4$, đa thức này bậc 1, nên j -**invariant** được xác định ngay lập tức: $j = 0$ và $j = 1728$ tương ứng. Đây là lý do tại sao secp256k1 ($j = 0$, $D = -3$) và nhiều đường cong BLS có dạng đơn giản $y^2 = x^3 + b$.

3.1.3. Quy trình CM đầy đủ

Cho bộ tham số (p, t, D) thỏa mãn $4p = t^2 + |D|v^2$:

1. **Tính j -invariant:** Tìm nghiệm $j_0 \in \mathbb{F}_p$ của $H_D(x) \pmod{p}$.
2. **Suy ra hệ số đường cong:** Từ j_0 , tính $c = j_0 \cdot (1728 - j_0)^{-1} \pmod{p}$, sau đó:
 - $a = 3c$, $b = 2c$ (trường hợp tổng quát $j_0 \neq 0, 1728$)
 - $a = 0$, b tùy ý (khi $j_0 = 0$, tức $D = -3$)
 - a tùy ý, $b = 0$ (khi $j_0 = 1728$, tức $D = -4$)
3. **Kiểm tra xoắn (Twist test):** Đường cong $E : y^2 = x^3 + ax + b$ và đường cong xoắn $E' : y^2 = x^3 + a\delta^2x + b\delta^3$ (với δ là phân tử không phải thặng dư bậc hai) có tổng số điểm $\#E + \#E' = 2(p + 1)$. Ta cần chọn đúng đường cong có $r \mid \#E$.
4. **Tìm điểm sinh (Generator):** Nhân ngẫu nhiên một điểm trên đường cong với hệ số cofactor $h = \#E/r$ để tìm điểm sinh G có bậc chính xác là r .

3.1.4. Trường hợp $D = -3$: Xoắn sextic

Với $D = -3$, tình huống phức tạp hơn vì đường cong $y^2 = x^3 + b$ có **6 xoắn** (sextic twists) thay vì chỉ 2. Sáu vết Frobenius khả dĩ là:

$$t, \quad -t, \quad \frac{t \pm 3v}{2}, \quad \frac{-t \pm 3v}{2}$$

trong đó $4p = t^2 + 3v^2$. Để xác định xoắn đúng, ta lần lượt thử các giá trị b nhỏ và kiểm tra số điểm thực sự của đường cong $y^2 = x^3 + b$ bằng cách tính phép nhân vô hướng $[p + 1 - t']P$ với điểm ngẫu nhiên P .

Trong cài đặt của luận văn, hàm `find_twist` thực hiện tính toán và lọc nghiệm từ 6 vết Frobenius này. Chi tiết mã nguồn tính toán mảng ứng viên được trình bày tại mã nguồn công khai của dự án [16].

3.2. Tham số đầu vào của hệ thống

Trước khi đi vào chi tiết thuật toán, phần này giải thích ý nghĩa và cơ sở lựa chọn của 7 tham số đầu vào trong cài đặt thực tế, đã được tổng hợp tại bảng đặc trưng ở cuối mục này.

3.2.1. $k = 18$ — Bậc nhúng (Embedding Degree)

Bậc nhúng k xác định trường mở rộng $\mathbb{F}_{p^k}$ nơi phép ghép cặp Weil/Tate được tính toán. Giá trị $k = 18$ được chọn vì:

- **Cân bằng bảo mật:** Với $r \approx 2^{512}$ và $p \approx 2^{1024}$, kích thước trường mở rộng là $p^k \approx 2^{1024 \times 18} = 2^{18432}$ bit. Theo tiêu chuẩn NFS hiện tại, bài toán DLP trên $\mathbb{F}_{p^{18}}$ ở mức bảo mật ≥ 256 bit — tương đương với mức bảo mật của khóa $r \approx 2^{512}$.
- **Hỗ trợ Đa thức Cyclotomic bậc 18:** $\Phi_{18}(T) = T^6 - T^3 + 1$ cho phép sinh r hiệu quả, có cấu trúc NTT-friendly tự nhiên khi $T \equiv 0 \pmod{2^k}$ (xem §3.3.1).
- **Thực tế:** $k = 18$ là bậc nhúng phổ biến trong họ đường cong KSS18 [19], phù hợp với mức bảo mật hậu lượng tử cao hơn các đường cong BLS12 (dùng trong Ethereum 2.0).

3.2.2. $d = 3$ — Biệt thức CM ($|D|$)

Biệt thức CM $D = -3$ được chọn vì những ưu điểm nổi bật:

- **Đa thức Hilbert bậc 1:** $H_{-3}(x) = x$, nên j -invariant bằng 0 và phương trình đường cong có dạng cực kỳ đơn giản $y^2 = x^3 + b$. Không cần giải đa thức bậc cao.

- **Phổ biến trong thực tế:** secp256k1 (Bitcoin), BLS12-381 (Ethereum), và nhiều đường cong quan trọng khác đều dùng $D = -3$.
- **Điều kiện CM đơn giản:** Phương trình $4p = t^2 + 3v^2$ dễ giải hơn với hầu hết các giá trị (t, v) nguyên, tăng xác suất tìm được p nguyên tố trong lưới nâng.

3.2.3. *target_r_bits = 512* — Kích thước bậc nhóm r

Tham số này xác định **mức bảo mật** của bài toán logarit rời rạc trên đường cong (ECDLP):

- Với $r \approx 2^{512}$, thuật toán Pollard-rho tốt nhất cần $\approx 2^{256}$ phép tính, đạt mức bảo mật **256-bit** — vượt tiêu chuẩn NIST SP 800-57 cho ứng dụng đến năm 2030+ và phù hợp với kích bản hậu lượng tử (post-quantum).
- Đây cũng là kích thước của **khóa riêng** (private key scalar) và **kích thước chữ ký** trong các giao thức Schnorr và ECDSA trên đường cong này.

3.2.4. *target_p_bits = 1024* — Kích thước trường cơ sở p

Kích thước trường cơ sở tác động đến hai yếu tố:

- **Bảo mật DLP trên $\mathbb{F}_{p^k}$:** Với $p \approx 2^{1024}$ và $k = 18$, kích thước trường mở rộng là 2^{18432} bit. Theo ước lượng NFS/TNFS, điều này cho mức bảo mật ≥ 250 bit — phù hợp với ứng dụng yêu cầu bảo mật dài hạn.
- **Hiệu năng:** Kích thước 1024 bit làm cho mỗi phép nhân trường tốn gấp đôi so với 512 bit về mặt thời gian CPU, nhưng vẫn trong giới hạn thực tế với phép nhân Montgomery đa độ chính xác.
- **Tỉ số ρ :** $\rho = \log_2 p / \log_2 r = 1024/512 = 2$. Đây là tỉ số điển hình của Cocks-Pinch (so với $\rho = 1.5$ của BLS12 hay $\rho = 1$ của BN).

3.2.5. *max_attempts = 100_000* — Số lần thử tối đa

Giới hạn vòng lặp tìm kiếm an toàn. Trong thực nghiệm, thuật toán CP cải tiến thường tìm được kết quả trong **100–6000 lần thử** (tùy cấu hình NTT), do đó 100_000 là giới hạn rất thoải mái đảm bảo không bao giờ bị timeout trong điều kiện bình thường. Nếu vượt giới hạn này (không bao giờ xảy ra trong thực tế), hệ thống báo lỗi thay vì chạy mãi mãi.

3.2.6. *min_scalar_two_adicity = 32* — Two-adicity tối thiểu của $r - 1$

Tham số này ràng buộc **cấu trúc NTT-friendly** của trường vô hướng $\mathbb{F}_r$:

$$2^{32} \mid (r - 1) \iff r = d \cdot 2^{32} + 1 \text{ với } d \text{ lẻ}$$

Ý nghĩa thực tế: trường $\mathbb{F}_r$ chứa một căn nguyên thủy 2^{32} -th của đơn vị, cho phép thực hiện NTT trên các đa thức bậc lên đến $2^{32} \approx 4 \times 10^9$. Đây là yêu cầu cốt lõi để tạo chứng minh ZK hiệu quả:

- Trong Groth16/PLONK, phép nhân đa thức $A(x) \cdot B(x)$ với mạch có 2^{20} ràng buộc cần NTT bậc $\geq 2^{21}$ — thỏa mãn với 2^{32} .
- BLS12-381 (chuẩn công nghiệp) có r two-adicity = 32; hệ thống này đạt ≥ 33 , **vượt chuẩn BLS12-381**.

Giá trị 32 đạt được **không cần rejection sampling** nhờ ràng buộc $T \equiv 0 \pmod{2^{11}}$ (vì $v_2(r - 1) = 3 \cdot v_2(T) \geq 3 \times 11 = 33 \geq 32$).

3.2.7. *min_base_two_adicity = 32 — Two-adicity tối thiểu của $p - 1$*

Tương tự, tham số này ràng buộc **cấu trúc NTT-friendly của trường cơ sở $\mathbb{F}_p$** :

$$2^{32} \mid (p - 1) \iff p = d \cdot 2^{32} + 1$$

Đây cũng là yêu cầu của thuật toán Miller trong phép ghép cặp: các biểu thức trung gian (line evaluations) được tính trong $\mathbb{F}_{p^k}$ và có thể được tăng tốc bằng NTT nếu p có cấu trúc NTT. Ngoài ra:

- $p = d \cdot 2^{32} + 1$ kháng TNFS tốt hơn p tùy ý (xem §3.3), do cấu trúc $p - 1$ không có nhân tử nhỏ ngẫu nhiên mà thuật toán này có thể khai thác.
- Giá trị này đạt được bằng cách mở rộng lưới nâng (h_t, h_y) trong hàm `try_lift_to_prime`.

3.2.8. *Tổng hợp: Đặc trưng đường cong mục tiêu*

Từ 7 tham số trên, đường cong được xây dựng có đặc trưng:

Bảng 3.2. Tổng hợp đặc trưng đường cong mục tiêu

Thuộc tính	Giá trị	Ý nghĩa
Phương trình	$y^2 = x^3 + b$	$D = -3, j = 0$, dạng tối giản
Bậc nhúng	$k = 18$	Phép ghép cặp trong $\mathbb{F}_{p^{18}}$

Thuộc tính	Giá trị	Ý nghĩa
Kích thước r	512 bit	Bảo mật ECDLP 256-bit
Kích thước p	1024 bit	Bảo mật DLP ≥ 250 -bit
Two-adicity r	≥ 33	NTT/FFT trên $\mathbb{F}_r$ đến bậc 2^{33}
Two-adicity p	≥ 32	NTT/FFT trên $\mathbb{F}_p$ đến bậc 2^{32}
Dạng r	$d \cdot 2^{33} + 1$	NTT-friendly scalar field
Dạng p	$d \cdot 2^{34} + 1$	NTT-friendly base field (kháng TNFS)

3.3. Thuật toán Cocks-Pinch truyền thống

3.3.1. Vấn đề: Xây dựng ngược từ r

Phương pháp CM truyền thống xuất phát từ p được chọn trước, đòi hỏi giải phương trình $4p = t^2 + |D|v^2$ — một bài toán biểu diễn số nguyên bởi dạng toàn phương (quadratic form) không hề dễ trong không gian lớn.

Thuật toán Cocks-Pinch (CP) [3] đảo chiều bài toán: **bắt đầu từ r rồi xây dựng p** . Đây là cách tiếp cận tự nhiên hơn khi mục tiêu là đảm bảo r có kích thước an toàn (ví dụ: 512 bit) với bậc nhúng k cho trước.

3.3.2. Điều kiện bậc nhúng

Bậc nhúng k của đường cong là **số nguyên dương nhỏ nhất** sao cho $r \mid p^k - 1$, hay tương đương $p \equiv 1 \pmod{r}$ khi $k = 1$, hoặc tổng quát hơn p là nghiệm của $\Phi_k(x) \equiv 0 \pmod{r}$. Điều kiện cụ thể: $r \mid \Phi_k(p)$, trong đó Φ_k là đa thức cyclotomic. Điều này có nghĩa là tồn tại u nguyên tố cùng nhau với k sao cho:

$$p \equiv \rho^i \pmod{r} \quad \text{với } \rho \text{ là nghiệm nguyên thủy của } \Phi_k(x) \equiv 0 \pmod{r}$$

3.3.3. Thuật toán CP cho $k = 18$

Với $k = 18$ và $D = -3$, bậc nhóm $r = \Phi_{18}(T) = T^6 - T^3 + 1$ với T nguyên tùy ý. Các bước:

Bước 1 — Sinh r từ T :

$$r = \Phi_{18}(T) = T^6 - T^3 + 1$$

Kiểm tra tính nguyên tố của r bằng thuật toán Miller-Rabin.

Bước 2 — Tính $\sqrt{-D} \pmod{r}$:

Cần $\beta = \sqrt{-3} \pmod{r}$. Do $r \equiv 1 \pmod{3}$ (do cấu trúc của Φ_{18}), phần tử này luôn tồn tại. Dùng thuật toán Tonelli–Shanks.

Bước 3 — Tính t_0, y_0 modulo r :

Với mỗi i thỏa $\gcd(i, k) = 1$ và $1 \leq i < k$:

$$t_0 = T^i + 1 \pmod{r}, \quad y_0 = \frac{t_0 - 2}{\beta} \pmod{r}$$

Bước 4 — Nâng (lift) lên số nguyên:

Tìm $t, y \in \mathbb{Z}$ với $t \equiv t_0 \pmod{r}$, $y \equiv y_0 \pmod{r}$ sao cho:

$$p = \frac{t^2 + 3y^2}{4}$$

là số nguyên và là số nguyên tố. Để làm điều này, thử các dịch chuyển nhỏ:

$$t = t_0 + h_t \cdot r, \quad y = y_0 + h_y \cdot r, \quad h_t, h_y \in \{-20, \dots, 20\}$$

cho đến khi $\frac{t^2 + 3y^2}{4}$ là số nguyên tố đúng kích thước mục tiêu.

Trong phần cài đặt hệ thống, hàm `try_lift_to_prime` thực hiện quá trình quét lưới hai chiều này để lọc ra các ứng viên thỏa mãn. Chi tiết mã nguồn của thiết kế vòng lặp quét lưới được đính kèm tại mã nguồn công khai của dự án [16].

3.3.4. Phân tích độ phức tạp trung bình

Mật độ số nguyên tố lân cận N là khoảng $1/\ln N$, tức $1/1024 \ln 2 \approx 1/709$. Với lưới $(h_t, h_y) \in [-20, 20]^2$ tạo ra $41 \times 41 = 1681$ ứng viên p mỗi lần, tỉ lệ thành công mỗi lần sinh r là khoảng $1681/709 \approx 2.4$ ứng viên p nguyên tố. Trong thực tế, thường cần khoảng **100–500 lần sinh r** để tìm được bộ tham số hợp lệ.

3.4. Thuật toán Cocks-Pinch Cải tiến

3.4.1. Động lực: Tấn công rây trường số thấp (TNFS)

Bảo mật của phép ghép cặp phụ thuộc vào độ khó của **bài toán logarit rời rạc** (DLP) trên trường mở rộng $\mathbb{F}_{p^k}$. Năm 2016, Barbulescu và Duquesne [4] chỉ ra rằng thuật toán **Tower Number Field Sieve (TNFS)** — một biến thể của NFS — có thể giảm đáng kể độ phức tạp tấn công DLP trên $\mathbb{F}_{p^k}$ khi k composite và p có cấu trúc đặc biệt. Điều này **phá vỡ ước lượng bảo mật ban đầu** của các đường cong như BN256 (từ ~128 bit xuống còn ~100 bit hiệu quả).

Để kháng lại TNFS, cần đảm bảo trường cơ sở của đường cong có **cấu trúc tốt** ở cả hai trường: trường vô hướng $\mathbb{F}_r$ (phục vụ tính toán ZK proof) và trường cơ sở $\mathbb{F}_p$ (phục vụ phép ghép cặp).

3.4.2. Yêu cầu NTT-friendly

Một số nguyên tố q được gọi là **NTT-friendly** (hoặc FFT-friendly) nếu $q - 1$ chia hết cho một lũy thừa đủ lớn của 2, tức là:

$$2^s \mid (q - 1) \quad \text{với } s \text{ đủ lớn}$$

Số s được gọi là **số hạng hai-adicity** (two-adicity) của q . Ý nghĩa: trường $\mathbb{F}_q$ chứa một căn nguyên thủy 2^s -th của đơn vị, cho phép áp dụng **Biến đổi Lý thuyết số (NTT)** — tức FFT trên trường hữu hạn — để nhân đa thức bậc cao trong $O(n \log n)$ thay vì $O(n^2)$. Đây là yếu tố *quyết định* hiệu năng trong:

- Sinh chứng minh ZK (Groth16, PLONK): tính $H(x) = (A(x) \cdot B(x) - C(x))/Z(x)$
- Các cam kết đa thức (KZG, FRI)
- Thuật toán MSM (Multi-scalar multiplication) biến thể NTT

Tương quan với TNFS: Một trường $\mathbb{F}_p$ với two-adicity cao có $p - 1 = d \cdot 2^s$, nghĩa là $p = d \cdot 2^s + 1$. Cấu trúc này khiến cho tháp trường (tower field extension) $\mathbb{F}_{p^k}$ có đặc điểm tốt hơn từ góc độ kháng TNFS: các thuật toán rây cần khai thác cấu trúc phân tích của $p - 1$ để tìm quan hệ, do đó khi $p - 1$ có cấu trúc **không phân tích tùy tiện** mà theo dạng chuẩn $d \cdot 2^s + 1$, sự khai thác thông tin này trở nên khó hơn.

3.4.3. Cải tiến 1: NTT-friendly r qua ràng buộc T

Nhận xét then chốt: Vì $r = \Phi_{18}(T) = T^6 - T^3 + 1$, ta có:

$$r - 1 = T^3(T^3 - 1)$$

Nếu $T \equiv 0 \pmod{2^k}$, thì $T^3 \equiv 0 \pmod{2^{3k}}$, và do đó:

$$v_2(r - 1) = v_2(T^3(T^3 - 1)) = 3k$$

(vì $T^3 \equiv 0 \pmod{2^{3k}}$ và $T^3 - 1 \equiv -1 \pmod{2}$ là lẻ).

Kết luận: Để đảm bảo $\text{two-adicity}(r - 1) \geq s$, chỉ cần **chọn** $T \equiv 0 \pmod{2^{\lceil s/3 \rceil}}$. Đây là ràng buộc *không cần rejection sampling* — mọi T thỏa mãn điều kiện này đều cho r với two-adicity ít nhất s .

Trong cài đặt:

```

// Chọn T là bội số của 2^ceil(s/3): đảm bảo two_adicity(r-1) >= s
let t_align = min_scalar_two_adicity.div_ceil(3);
let step = U1024::ONE.shl(t_align as usize);
// t_val luôn là bội số của step
let t_val = t_base + U1024::rand(&t_steps) * step;
let r = cyclotomic_phi18(&t_val);
// Đảm bảo r thỏa mãn: two_adicity(r-1) = 3 * v_2(T) >= s

```

Mã nguồn 3.1. Cài đặt ràng buộc T để đạt two-adicity cho r

Với $\text{min_scalar_two_adicity} = 32$, ta cần $\lceil 32/3 \rceil = 11$, tức $T \equiv 0 \pmod{2^{11}}$. Điều này **không làm giảm không gian tham số** đáng kể — vẫn còn $2^{85}/2^{11} = 2^{74}$ ứng viên T trong dải hợp lệ.

3.4.4. Cải tiến 2: NTT-friendly p qua mở rộng lưới nâng

Sau khi có $t_0, y_0 \pmod{r}$, tham số p được xác định bởi:

$$p = \frac{(t_0 + h_t r)^2 + 3(y_0 + h_y r)^2}{4}$$

Vì $r \equiv 0 \pmod{2^{3k}}$ (do $T \equiv 0 \pmod{2^k}$), các bit thấp hơn $3k$ của $p-1$ chỉ phụ thuộc vào t_0, y_0 (cố định cho mỗi r). Tuy nhiên, các bit từ vị trí $3k$ trở lên bị ảnh hưởng bởi h_t, h_y . Để đạt $\text{two_adicity}(p-1) \geq s_p > 3k$, ta cần mở rộng lưới (h_t, h_y) ra ngoài phạm vi ± 20 mặc định:

$$\text{extra} = \max(0, s_p - 3k), \quad \text{half_range} = 20 + 2^{\text{extra}}$$

Điều này đảm bảo có đủ ứng viên h_t, h_y để **bao phủ mọi tổ hợp bit từ vị trí $3k$ đến s_p** trong $p-1$.

Quá trình tính toán động độ mở rộng của lưới tỷ lệ thuận với khoảng cách giữa mục tiêu bậc hai-adicity (two-adicity) của trường cơ sở so với mặc định. Chi tiết cấu trúc dữ liệu `LiftParams` và thuật toán tính toán độ mở rộng này được trình bày tại mã nguồn công khai của dự án [16].

3.4.5. Phân tích hiệu năng của thuật toán cải tiến

Bảng dưới so sánh thuật toán CP gốc và CP Cải tiến với cùng mục tiêu ($r \approx 2^{512}, p \approx 2^{1024}, k = 18$):

Bảng 3.3. So sánh thuật toán CP truyền thống và CP Cải tiến

Tham số	CP Truyền thống	CP Cải tiến ($s_r = 32, s_p = 32$)
Ràng buộc trên T	Không có	$T \equiv 0 \pmod{2^{11}}$
two-adicity của $r - 1$	Ngẫu nhiên (~1-3)	≥ 33 (đảm bảo)
two-adicity của $p - 1$	Ngẫu nhiên (~1)	≥ 32 (với lưới mở rộng)
Lưới (h_t, h_y)	$[-20, 20]^2$	$[-21, 21]^2$ (khi $s_p \leq 3k$)
Chi phí mỗi lần thử	~50ms	~50ms
Số lần thử trung bình	~100–500	~3.000–6.000
Thời gian thực tế	~2–10 giây	~30–60 giây
r hỗ trợ NTT?	Không	Có (bậc 2^{33})
p hỗ trợ NTT?	Không	Có (bậc 2^{34})

Kết quả thực nghiệm điển hình:

r two-adicity: $2^{33} \mid (r-1) \rightarrow r = d \cdot 2^{33} + 1$, NTT đến bậc 2^{33}
 p two-adicity: $2^{34} \mid (p-1) \rightarrow p = d \cdot 2^{34} + 1$, NTT đến bậc 2^{34}
 Thời gian tổng: 32.81s

3.4.6. So sánh với BLS12-381

Đường cong BLS12-381 [20] — chuẩn công nghiệp hiện tại trong ZK proof — có: - r two-adicity = **32** (đủ cho ZK circuits đến 2^{32} ràng buộc) - p two-adicity = **32**

Đường cong được xây dựng trong luận văn này đạt **33–34**, tức là **vượt BLS12-381 về cấu trúc NTT** trong khi kích thước khóa lớn hơn ($p \approx 2^{1024}$ so với $p \approx 2^{381}$), cung cấp mức bảo mật hậu lượng tử cao hơn trong ngữ cảnh phép ghép cặp.

3.4.7. Giới hạn và hướng mở rộng

Thuật toán cải tiến vẫn có giới hạn thực tế:

- **Two-adicity tối đa đạt được:** Khi $s_p > 3k + 10$, lưới (h_t, h_y) phải có $\text{half_range} > 2^{10} = 1024$, số lần lặp vượt 10^6 mỗi ứng viên r , khiến tìm kiếm không thực tế. Nếu cần s_p cực cao (> 50), cần dùng họ đường cong chuyên dụng thiết kế từ đầu như Pasta curves (Pallas/Vesta).
- **Tỉ số ρ :** Thuật toán CP vốn cho tỉ số $\rho = \log p / \log r \approx 2$, không tối ưu bằng các họ đường cong chuyên biệt như BN256 ($\rho = 1$). Đây là sự đánh đổi cố hữu của phương pháp CP.

- **Chỉ** $k = 18$ **và** $D = -3$ được cài đặt trong hệ thống hiện tại. Mở rộng sang $k = 12, 24$ đòi hỏi thay đổi đa thức cyclotomic và công thức CM.

CHƯƠNG 4

SƠ ĐỒ CHỮ KÝ SỐ

Chương này trình bày ba sơ đồ chữ ký số được xây dựng trên nền tảng đường cong elliptic đã sinh ở Chương 3: **Schnorr**, **ECDSA**, và **chữ ký tổng hợp BLS**. Với mỗi sơ đồ, chúng ta đi qua lý thuyết cốt lõi, phân tích cấu trúc dữ liệu và cài đặt thực tế bằng Rust. Kết quả đo hiệu năng và phân tích bảo mật được trình bày riêng tại **Chương 5**.

4.1. Cặp khóa (KeyPair)

Mọi sơ đồ chữ ký đều dùng chung cấu trúc cặp khóa:

```
pub struct KeyPair<C: SWCurveConfig> {  
    pub private_key: U1024,           //  $d$  in  $[1, r-1]$ , bí mật  
    pub public_key: AffinePoint<C>, //  $Q = [d]G$ , công khai  
}
```

Mã nguồn 4.1. Cấu trúc dữ liệu cặp khóa KeyPair

- **Khóa riêng** d : một số nguyên ngẫu nhiên trong $[1, r - 1]$, sinh bởi CSPRNG (`U1024::rand`).
- **Khóa công khai** $Q = [d]G$: tính bằng phép nhân vô hướng. Với $r \approx 2^{512}$, tính Q từ d là tính khả thi (mili giây), nhưng bài toán ngược (tính d từ Q) là ECDLP — độ phức tạp $O(2^{256})$.

Cặp khóa được lưu và đọc từ file nhị phân 128 byte (big-endian) với quyền truy cập khắt khe 0600 trên hệ điều hành Unix phân quyền để giảm rủi ro bị đọc lên bởi các tài khoản người dùng khác trên cùng máy chủ. Các thao tác khởi tạo (`generate`) ngẫu nhiên và truy xuất dữ liệu tĩnh (`save`) được xử lý bằng API tiêu chuẩn. Chi tiết mã nguồn quản lý vòng đời cặp khóa được đính kèm tại mã nguồn công khai của dự án [16].

4.1.1. Hàm băm mở rộng

Vì $r \approx 2^{512}$ và SHA-256 chỉ cho 256 bit, hệ thống dùng **4 lần SHA-256** với tiền tố khác nhau để sinh 1024 bit, rồi cắt lấy 512 bit thấp khi cần:

Để đáp ứng yêu cầu kích thước, hàm `hash_message` được cấu hình để thực hiện băm lặp chuỗi đầu vào theo các bộ tiền tố biến thiên. Cấu trúc lặp này vận hành tương đương

như một hàm **SHA-1024** tùy chỉnh. Sự thiết kế này đảm bảo vùng không gian đầu ra đủ độ rộng nhằm phòng tránh hiệu quả các biến thể độ vòng lặp có nghĩa với mức bảo mật $r \approx 2^{512}$. Chi tiết cài đặt thuật toán kết hợp băm mở rộng được cung cấp tại mã nguồn công khai của dự án [16].

4.2. Chữ ký Schnorr

4.2.1. Lý thuyết

Schnorr là sơ đồ chữ ký Sigma-protocol dựa trên giả thuyết logarit rời rạc (DL) [13]. Nó được ưa chuộng vì độ đơn giản của chứng minh bảo mật (trong mô hình Oracle ngẫu nhiên) và hỗ trợ tổng hợp tự nhiên.

Ký: Cho khóa riêng $d \in [1, r - 1]$ và thông điệp m :

1. Chọn ngẫu nhiên $k \xleftarrow{R} [1, r - 1]$
2. Tính điểm $R = [k]G$
3. Tính thách thức $e = H(R_x \| R_y \| m) \pmod{r}$
4. Tính $s = k + e \cdot d \pmod{r}$
5. Chữ ký: $\sigma = (R, s)$

Xác minh: Cho khóa công khai Q và chữ ký (R, s) :

1. Tính $e = H(R_x \| R_y \| m)$
2. Kiểm tra $[s]G = R + [e]Q$

Tính đúng đắn: $[s]G = [k + ed]G = [k]G + [e][d]G = R + [e]Q$ [ok]

4.2.2. Cấu trúc dữ liệu

```
pub struct SchnorrSignature<C: SWCurveConfig> {
    pub r_point: AffinePoint<C>, // R = [k]G (điểm 256 byte: 2x128 byte tọa độ)
    pub s:      U1024,           // s in [0, r-1], 128 byte
}
```

Mã nguồn 4.2. Cấu trúc dữ liệu chữ ký Schnorr

Kích thước chữ ký: $2 \times 128 + 128 = 384$ byte (R gồm hai tọa độ x, y mỗi tọa độ 128 byte, cộng thêm s 128 byte).

4.2.3. Cài đặt

Các bước ký và xác minh thuật toán Schnorr được nhóm tác giả cài đặt bám sát với quy trình lý thuyết toán học (đã trình bày ở Mục 4.2.1), tận dụng triệt để kiến trúc cấp phát và an toàn bộ nhớ của Rust để loại bỏ lỗi tràn giới hạn (buffer overflow) sinh ra từ việc xử lý chuỗi số lớn. Thuật toán cài đặt tích hợp hàm băm nội sinh `challenge_hash` dùng nối trực tiếp các thành phần R_x, R_y, m trước khi gọi hàm băm mở rộng. Quy trình gộp này đảm bảo “thách thức” sinh ra phụ thuộc định danh vào toàn bộ ngữ cảnh thông điệp, triệt tiêu mọi rủi ro tấn công giả mạo (forgery) bóc tách dữ liệu — đặc biệt theo kiểu công kích Pohlig-Hellman trên phần tử nonce.

Chi tiết mã nguồn của các hàm `sign()` và `verify()` thuộc thuật toán Schnorr được đính kèm tại mã nguồn công khai của dự án [16].

4.2.4. Bảo mật

- **Unforgeability (EUF-CMA):** Chứng minh được trong Random Oracle Model dưới giả thuyết DL. Kẻ tấn công cần giải ECDLP hoặc bẻ hàm băm.
- **Nguy cơ nonce tái sử dụng:** Nếu k bị tái sử dụng cho hai thông điệp khác nhau, kẻ công phá có thể tính $d = (s_1 - s_2)/(e_1 - e_2) \pmod{r}$. Cài đặt dùng CSPRNG (`rand::thread_rng`) cho mỗi lần ký.

4.3. Chữ ký ECDSA

4.3.1. Lý thuyết

ECDSA (Elliptic Curve Digital Signature Algorithm) là chuẩn IEEE P1363 và FIPS 186-4 [12]. Nó khác Schnorr ở chỗ r trong chữ ký là **tọa độ x của điểm R** lấy modulo n , thay vì bản thân điểm R .

Ký: Cho khóa riêng d và thông điệp m :

1. $k \xleftarrow{R} [1, r - 1]$
2. $R = [k]G$
3. $r_{\text{sig}} = R_x \pmod{n}$ (nếu $r_{\text{sig}} = 0$, thử lại)
4. $e = H(m)$
5. $s = k^{-1}(e + r_{\text{sig}} \cdot d) \pmod{n}$ (nếu $s = 0$, thử lại)
6. Chữ ký: $\sigma = (r_{\text{sig}}, s)$

Xác minh:

1. Kiểm tra $r_{\text{sig}}, s \in [1, n - 1]$

2. $e = H(m), w = s^{-1} \pmod n$
3. $u_1 = ew, u_2 = r_{\text{sig}} \cdot w$, cả hai tính mod n
4. $P = [u_1]G + [u_2]Q$
5. Hợp lệ khi $P \neq \mathcal{O}$ và $P_x \pmod n = r_{\text{sig}}$

4.3.2. Cấu trúc dữ liệu

```
pub struct EcdsaSignature {
    pub r: U1024, // r_sig = R.x mod n, 128 byte
    pub s: U1024, // s in [1, n-1], 128 byte
}
```

Mã nguồn 4.3. Cấu trúc dữ liệu chữ ký ECDSA

Kích thước chữ ký: $128 + 128 = 256$ byte — nhỏ hơn Schnorr vì không lưu điểm R đầy đủ.

4.3.3. Cài đặt

Tương tự Schnorr, các thủ tục ký xác thực và kiểm chứng chuẩn mực của thuật toán ECDSA cũng được triển khai trong không gian số học bằng các thao tác đại số trực tiếp trên cấu trúc thành phần `FieldElement`. Tại quy trình ký sinh, tiến trình tạo phần tử ngẫu nhiên nonce được rà soát khắt khe và lập tức phải loại bỏ (chu kỳ thử lại mới) nếu tổ hợp tạo ra vi phạm hệ số tiệm cận, cụ thể rơi vào 2 trường hợp suy biến tiềm ẩn: $r_{\text{sig}} = 0$ hoặc $s = 0$. Logic bóc lỗi này tuân thủ trọn vẹn đặc tả bảo mật thuộc tiêu chuẩn chuẩn hóa FIPS 186-4 được ban hành riêng cho ECDSA.

Chi tiết mã nguồn của các hàm `sign()` và `verify()` thuộc thuật toán ECDSA được đính kèm tại mã nguồn công khai của dự án [16].

4.3.4. So sánh Schnorr và ECDSA

Bảng 4.1. So sánh sơ đồ chữ ký Schnorr và ECDSA

Tiêu chí	Schnorr	ECDSA
Chuẩn hóa	ISO/IEC 14888-3	FIPS 186-4, IEEE P1363
Kích thước chữ ký	384 byte ($2 \times 128 + 128$)	256 byte (2×128)
Phép nhân vô hướng ký	1	1
Phép nhân vô hướng xác minh	2 (tuần tự)	2 (tuần tự)

Tiêu chí	Schnorr	ECDSA
Bảo mật chứng minh	RO Model, DL	Heuristic, DL + hash
Hỗ trợ tổng hợp	Có (MuSig, FROST)	Không
Rủi ro nonce tái sử dụng	Lộ khóa riêng	Lộ khóa riêng

4.4. Chữ ký tổng hợp BLS

4.4.1. Lý thuyết (tổng quan)

Chữ ký BLS (Boneh-Lynn-Shacham, 2001) [21] là sơ đồ **tổng hợp phi tương tác**: n người ký với n khóa riêng khác nhau có thể tạo ra **một chữ ký duy nhất** đại diện cho tất cả, và xác minh chỉ cần một phép ghép cặp thay vì n phép.

Nền tảng: Dùng phép ghép cặp song tuyến $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ với:

- $\mathbb{G}_1 = E(\mathbb{F}_p)$, $\mathbb{G}_2 = E'(\mathbb{F}_{p^2})$ (hoặc tương đương)
- $\mathbb{G}_T \subset \mathbb{F}_{p^k}^*$

Ký: Người thứ i với khóa riêng d_i :

$$\sigma_i = [d_i]H(m) \in \mathbb{G}_1$$

trong đó $H : \{0, 1\}^* \rightarrow \mathbb{G}_1$ là hàm băm lên nhóm điểm.

Tổng hợp: $\sigma = \sigma_1 + \sigma_2 + \dots + \sigma_n$ (phép cộng điểm, không cần tương tác)

Xác minh tổng hợp:

$$e(\sigma, G_2) = e(H(m), Q_1 + Q_2 + \dots + Q_n)$$

trong đó $Q_i = [d_i]G_2$ là khóa công khai của người i .

Tính đúng đắn:

$$e\left(\sum_i [d_i]H(m), G_2\right) = e\left(H(m), \sum_i [d_i]G_2\right) = e(H(m), \sum Q_i) \checkmark$$

4.4.2. Vai trò của đường cong KSS18

Đường cong được xây dựng trong luận văn này (họ Kachisa-Schaefer-Scott với $k = 18$) [19] cung cấp nền tảng để cài đặt BLS:

- $\mathbb{G}_1 \subset E(\mathbb{F}_p)$: nhóm điểm trên trường cơ sở, kích thước r .

- $\mathbb{G}_T \subset \mathbb{F}_{p^{18}}^*$: nhóm đích của phép ghép cặp, bậc r , kích thước trường 18432 bit — đủ để kháng cả NFS lẫn TNFS.

Phép ghép cặp Optimal Ate (trên KSS18) chạy vòng lặp Miller với $O(\log r)$ bước, mỗi bước thực hiện phép tính trong $\mathbb{F}_{p^{18}}$. Cấu trúc NTT-friendly của p (two-adicity $(p-1) \geq 32$) giúp tăng tốc các phép nhân trong tháp trường.

Lưu ý: Cài đặt phép ghép cặp đầy đủ (vòng lặp Miller + phép lũy thừa lần cuối) nằm ngoài phạm vi của luận văn này và là hướng phát triển tiếp theo. Chương này tập trung vào Schnorr và ECDSA trên $\mathbb{G}_1$.

4.5. Cài đặt hệ thống ký (**curve1024-sig**)

Binary `curve1024-sig` cung cấp giao diện dòng lệnh kiểu GPG cho toàn bộ vòng đời khóa:

```
curve1024-sig keygen [-o <file>]
curve1024-sig sign -f <file> [-k <key>] [--scheme schnorr|ecdsa]
curve1024-sig verify -f <file> [-k <pub>]
curve1024-sig export-pub [-k <priv>] [-o <pub>]
```

Mã nguồn 4.4. Giao diện dòng lệnh CLI `curve1024-sig`

Các tham số được đọc từ `config/curve1024.toml` thông qua `build.rs` để sinh hằng số tĩnh — đảm bảo zero-cost abstraction và không có chi phí khởi tạo runtime.

CHƯƠNG 5

CÀI ĐẶT VÀ ĐÁNH GIÁ

Chương này đánh giá toàn diện hệ thống đã xây dựng, không chỉ dừng lại ở trình bày số liệu mà tập trung **chứng minh giá trị** của hệ thống trên hai phương diện: hiệu năng thực tế và độ an toàn mật mã. Mục tiêu trả lời câu hỏi: *Các con số đo được nói lên điều gì? Hệ thống này có an toàn không? Có dùng được trong thực tế không?*

5.1. Đánh giá hiệu năng

5.1.1. Kết quả đo đạc

Toàn bộ hệ thống được đo bằng framework Criterion (release build, tối ưu LLVM O3, 10 mẫu/phép đo, phần cứng thông thường không có AVX-512). Kết quả cụ thể như sau:

Bảng 5.1. Kết quả đo hiệu năng hệ thống Curve1024

Phép đo	Min	Mean	Max
KeyPair::generate	1.629 s	1.687 s	1.757 s
Schnorr/sign	1.473 s	1.551 s	1.641 s
Schnorr/verify	2.656 s	2.831 s	3.025 s
ECDSA/sign	1.524 s	1.606 s	1.711 s
ECDSA/verify	2.847 s	3.140 s	3.459 s
Nhân vô hướng $[k]G$	1.061 s	~1.1–1.3 s	1.489 s

5.1.2. Phân tích: Kết quả có hợp lý về mặt lý thuyết không?

Trước tiên cần nhìn nhận rằng Curve1024 sử dụng trường nguyên tố p **1024-bit** — gấp 4 lần kích thước của secp256k1 (256-bit) hay P-384. Đây là lựa chọn chủ động, không phải sai sót thiết kế, vì mục tiêu là bảo mật 256-bit và tính pairing-friendly. Hệ quả tất yếu là hiệu năng chậm hơn đáng kể.

Phép nhân vô hướng dùng thuật toán Double-and-Add với **1024 vòng lặp** (tương ứng kích thước bit của trường vô hướng $r = 512$ bit — số vòng lặp bằng độ dài bit của trường cơ sở p). Mỗi vòng lặp thực hiện một phép `double` và có thể một phép `add`, mỗi phép toán đó cần **1 phép nghịch đảo modulo p** trong hệ tọa độ Affine hiện tại. Phép nghịch đảo được tính bằng định lý Fermat: $a^{-1} = a^{p-2} \pmod{p}$, tức là gọi đệ quy `pow` với

khoảng **~1536 phép nhân Montgomery** 1024-bit bên trong. Đây là nguồn gốc của con số ~1.1–1.6 giây/phép nhân vô hướng.

Schnorr ký nhanh hơn ECDSA ~3% do Schnorr không cần tính nghịch đảo $k^{-1} \pmod{r}$ trong bước sinh chữ ký. Xác minh (cả Schnorr và ECDSA) tốn gần gấp đôi ký do cần **2 phép nhân vô hướng độc lập** ($[s]G$ và $[e]Q$ cho Schnorr; $[u_1]G$ và $[u_2]Q$ cho ECDSA). Tỷ lệ này hoàn toàn phù hợp với dự kiến lý thuyết.

So với secp256k1, độ chênh lệch (~1.5 s so với ~0.2 ms) xuất phát chủ yếu từ ba yếu tố tích lũy: (1) scalar dài gấp 4 lần kéo dài thêm $4\times$ số vòng lặp, (2) mỗi phép nhân Montgomery trên số 1024-bit tốn ~16 lần công sức so với 256-bit (chi phí $O(n^2)$ với n số limb), và (3) secp256k1 có thư viện tối ưu nhiều năm với assembly AVX-512 được tinh chỉnh. Kết hợp lại, khoảng cách ~8000 $\times$ là kỳ vọng hợp lý, không phải dấu hiệu của thuật toán sai.

5.1.3. Tính khả thi trong ứng dụng thực tế

Với thời gian ~1.5–1.7 giây cho một phép ký, hệ thống hoàn toàn phù hợp với các **kịch bản ký ngoại tuyến** như ký tài liệu pháp lý, ký gói phần mềm khi phát hành, ký giao dịch tài chính giá trị lớn, hay ký chứng chỉ trong hạ tầng PKI — những trường hợp mà độ trễ vài giây là chấp nhận được và bù đắp hoàn toàn bởi biên bảo mật 256-bit so với 128-bit của các chuẩn thông thường.

Ngược lại, hệ thống **chưa phù hợp** cho các ứng dụng yêu cầu độ trễ dưới mili-giây như TLS handshake xử lý hàng nghìn kết nối/giây, giao dịch thanh toán tốc độ cao, hay ký batch thời gian thực. Đây là sự đánh đổi có chủ ý — tập trung vào tính đúng đắn và bảo mật của nền tảng toán học trước, tối ưu hóa sau.

Nhờ áp dụng **phép nhân Montgomery** thay cho phép chia lấy dư thông thường, tất cả phép nhân trong $\mathbb{F}_p$ đều thực thi bằng phép shift và trừ có điều kiện, không cần chia số 1024-bit cho p — một phép toán đắt hơn nhân ~5–10 lần. Đây là tối ưu quan trọng nhất đã được triển khai trong cài đặt hiện tại.

Các hướng cải tiến hiệu năng tiếp theo được phân tích tại Chương 6.

5.2. Đánh giá độ an toàn

5.2.1. An toàn trước tấn công cổ điển

Hệ thống sử dụng trường nguyên tố p có độ lớn 1024-bit và bậc nhóm r là 512-bit. Bài toán ECDLP trên nhóm $E(\mathbb{F}_p)$ — tức là tìm k từ $Q = [k]G$ — hiện chưa có thuật toán tốt hơn thuật toán tổng quát. Thuật toán tốt nhất hiện tại là **Pollard's ρ** [9], có độ phức tạp $O(\sqrt{r}) \approx O(2^{256})$ phép toán nhóm. Với 2^{256} bước tính toán cần thực hiện, ngay cả khi

huy động toàn bộ năng lực tính toán của nhân loại — mỗi nguyên tử Trái Đất thực hiện 10^9 phép tính mỗi giây từ khi Big Bang — vẫn không đủ để phá vỡ khóa trong tuổi thọ của vũ trụ. Đây là **biên an toàn tuyệt đối** trước mọi hệ thống máy tính cổ điển.

Mức bảo mật 256-bit vượt xa chuẩn tối thiểu NIST khuyến nghị (128-bit cho các ứng dụng đến năm 2030) và tương đương với AES-256 — chuẩn mã hóa đối xứng mạnh nhất hiện nay. Đây là mục tiêu tường minh của thuật toán Cocks-Pinch cải tiến được trình bày tại Chương 3.

5.2.2. An toàn trước tấn công *Invalid Curve*

Invalid Curve Attack là một dạng tấn công nguy hiểm trong ECC: kẻ tấn công gửi một điểm P' không thuộc đường cong E nhưng vẫn hợp lệ về mặt định dạng, ép hệ thống tính toán trên một đường cong suy biến có bậc nhỏ, từ đó thu thập thông tin về khóa riêng qua nhiều truy vấn.

Hệ thống hiện tại **miễn nhiễm hoàn toàn** với tấn công này. Mọi điểm được tạo ra thông qua `AffinePoint::new(x, y)` đều tự động kiểm tra điều kiện:

$$y^2 \equiv x^3 + b \pmod{p}$$

bằng lời gọi `assert!(self.is_on_curve())` ngay trong hàm khởi tạo. Không tồn tại đường dẫn code nào cho phép một điểm không hợp lệ tồn tại trong hệ thống. Điều này được xác nhận bởi test `test_curve_point` trong bộ kiểm thử đơn vị.

5.2.3. An toàn trước tấn công *MOV*

Tấn công MOV (Menezes-Okamoto-Vanstone) [22] sử dụng phép ghép cặp (pairing) để ánh xạ bài toán ECDLP trong $E(\mathbb{F}_p)$ về bài toán DLP trong trường hữu hạn mở rộng $\mathbb{F}_{p^k}$, nơi bài toán DLP có thể được giải hiệu quả hơn bằng thuật toán chỉ số (index calculus). Tuy nhiên, hiệu quả của tấn công này phụ thuộc vào việc $\mathbb{F}_{p^k}$ có đủ nhỏ để bài toán DLP khả thi hay không. Với Curve1024, bậc nhúng $k = 18$, trường mở rộng $\mathbb{F}_{p^{18}}$ có kích thước $|p^{18}| = 18 \times 1024 = 18432$ bit. Bài toán DLP trong trường 18432-bit hoàn toàn không thể giải được với công nghệ hiện tại — trên thực tế, kinh nghiệm cộng đồng yêu cầu $k|p| \geq 3072$ bit để kháng tấn công này ở mức 128-bit [4]; hệ thống đạt $18 \times 1024 = 18432$ bit, vượt xa tiêu chuẩn này. Kết quả này được test `test_mov_attack_resistance` xác nhận tường minh.

5.2.4. An toàn trước tấn công *Anomalous*

Tấn công Anomalous (SSSA) [23] khai thác trường hợp đặc biệt khi $\#E(\mathbb{F}_p) = p$, tức là số điểm trên đường cong bằng đúng modulus. Trong trường hợp này, tồn tại một đẳng cấu nhóm ánh xạ bài toán ECDLP về bài toán trên nhóm cộng $\mathbb{Z}/p\mathbb{Z}$, cho phép giải trong thời gian tuyến tính.

Curve1024 có $\#E(\mathbb{F}_p) = h \cdot r$ với r là bậc nhóm con nguyên tố 512-bit và h là cofactor, trong khi p là số nguyên tố 1024-bit. Hiển nhiên $\#E(\mathbb{F}_p) \neq p$ vì $|r| = 512 < 1024 = |p|$, nên điều kiện anomalous không thoả mãn. Test `test_anomalous_attack_resistance` xác minh điều này.

5.2.5. Hạn chế: *Timing Side-Channel Attack*

Đây là điểm cần trung thực học thuật: thuật toán **Double-and-Add** hiện tại, mặc dù có `conditional_select` ở từng bước đơn lẻ, vẫn có tổng số phép `add` thực hiện **phụ thuộc vào số lượng bit 1 trong scalar** k . Kẻ tấn công có thể đo thời gian thực thi để suy luận về trọng số Hamming của khóa riêng, từ đó thu hẹp không gian tìm kiếm.

Đây là sự đánh đổi có chủ ý để giữ kiến trúc đơn giản (nguyên tắc KISS — Keep It Simple and Secure). Biện pháp khắc phục là thay thế bằng **Montgomery Ladder**, thuật toán thực hiện đúng 1024 phép `double` và 1024 phép `add` bất kể giá trị scalar, đảm bảo tính *constant-time* thực sự. Đây là hướng phát triển ưu tiên cao.

Toàn bộ kết quả kiểm thử **36/36 test PASSED** (17 đơn vị + 19 tích hợp) xác nhận tính đúng đắn toán học và an toàn của hệ thống trên mọi tiêu chí đã mô tả.

CHƯƠNG 6

KẾT LUẬN

6.1. Kết quả đạt được

Khóa luận đã hoàn thành mục tiêu đề ra: thiết kế và cài đặt từ đầu (from scratch) một hệ thống mật mã đường cong elliptic hoàn chỉnh trên trường nguyên tố 1024-bit bằng ngôn ngữ Rust, không phụ thuộc bất kỳ thư viện mật mã bên ngoài nào. Các đóng góp cụ thể bao gồm:

- **Lớp số học bignum an toàn bộ nhớ:** Tự định nghĩa kiểu dữ liệu $U1024$ với phép cộng, trừ, nhân, lũy thừa và nghịch đảo. Phép nhân Montgomery [17] được cài đặt trực tiếp, loại bỏ hoàn toàn phép chia số lớn trong vòng lặp tính toán, đồng thời bảo đảm an toàn bộ nhớ tuyệt đối nhờ kiến trúc Rust.
- **Đường cong pairing-friendly mới 1024-bit:** Áp dụng thành công Phương pháp Nhân phức (CM) và thuật toán Cocks-Pinch cải tiến [3] để chủ động sinh đường cong KSS18 với bậc nhúng $k = 18$, bậc nhóm $r \approx 2^{512}$ (bảo mật 256-bit), và cấu trúc NTT-friendly trên cả hai trường $\mathbb{F}_r$ và $\mathbb{F}_p$. Đây là đóng góp gốc của khóa luận — đường cong tự sinh, không lấy từ bất kỳ tiêu chuẩn nước ngoài nào, đồng thời kháng được tấn công TNFS [4], MOV [22], và Anomalous [23].
- **Ba sơ đồ chữ ký số chuẩn mực:** Schnorr [13] và ECDSA [12] được cài đặt đầy đủ và xác minh đúng đắn. Nền tảng lý thuyết của BLS [21] được trình bày và tích hợp vào kiến trúc, sẵn sàng cho cài đặt phép ghép cặp đầy đủ trong tương lai.
- **Bộ kiểm thử toàn diện:** 36/36 test PASSED, bao phủ từ số học cấp thấp ($U1024$, `FieldElement`) đến an toàn mật mã cấp cao (mô phỏng tấn công MOV, Anomalous, Invalid Curve).

6.2. Đánh giá tổng quan

6.2.1. So sánh với mục tiêu ban đầu

Nhìn lại bốn mục tiêu đặt ra tại Chương 1, khóa luận đạt được kết quả như sau:

Bảng 6.1. Đối chiếu kết quả với mục tiêu đề ra

Mục tiêu	Kết quả
Tự chủ tham số đường cong	✓ Sinh được đường cong KSS18 1024-bit mới, không tái sử dụng hằng số bên ngoài
Bảo mật trước máy tính cổ điển	✓ Bảo mật 256-bit, vượt xa chuẩn NIST 128-bit đến năm 2030
An toàn bộ nhớ ngôn ngữ	✓ Rust loại bỏ toàn bộ lỗi bộ nhớ tại thời điểm biên dịch
Chữ ký số vận hành được	✓ Schnorr và ECDSA ký/xác minh chính xác, 36/36 test PASSED

6.2.2. Đánh giá điểm mạnh và hạn chế

Điểm mạnh:

Hệ thống đạt được tính *đúng đắn toán học* cao — mọi công thức từ phép cộng điểm affine, phép nhân Montgomery đến quy trình ký/xác minh Schnorr và ECDSA đều được kiểm thử độc lập. Tính *minh bạch thuật toán* là ưu điểm nổi bật so với các thư viện “hộp đen” công nghiệp: toàn bộ tham số đường cong sinh ra từ quy trình Cocks-Pinch xác định, không có hạt giống (seed) bí ẩn, đảm bảo không thể tồn tại backdoor ẩn.

Hạn chế:

Hiệu năng hiện tại (~1.5 s/phép ký) chưa đủ cho các ứng dụng yêu cầu độ trễ thấp như TLS hay thanh toán thời gian thực. Đây là sự đánh đổi có chủ ý khi ưu tiên tính đúng đắn và kiến trúc phân tầng rõ ràng trước, tối ưu hóa sau. Ngoài ra, thuật toán Double-and-Add hiện tại tiềm ẩn nguy cơ timing side-channel do số phép add phụ thuộc vào trọng số Hamming của scalar — một điểm yếu cần giải quyết trước khi đưa vào môi trường production.

6.3. Hướng phát triển

6.3.1. Tối ưu hóa hiệu năng (ngắn hạn)

1. **Tọa độ Jacobian/Projective:** Loại bỏ phép nghịch đảo tốn kém ($a^{p-2} \bmod p$) trong mỗi bước add/double bằng cách biểu diễn điểm ở dạng $(X : Y : Z)$ thay vì (x, y) Affine. Toàn bộ vòng lặp nhân vô hướng 1024 bước chỉ cần 1 phép nghịch đảo duy nhất ở cuối — ước tính giảm thời gian ký từ ~1.5 s xuống dưới **200 ms**.

2. **Sliding Window / NAF:** Giảm ~25–30% số phép `add` so với Double-and-Add cơ bản bằng cách nhóm nhiều bit scalar lại. Kết hợp với Jacobian, thời gian ký dự kiến đạt dưới **150 ms**.
3. **AVX-512 / SIMD assembly:** Vector hóa các phép nhân 64×64-bit limb trong Montgomery multiplication trên kiến trúc x86-64. Thư viện `blst` của Ethereum đạt tốc độ BLS12-381 signing dưới 1 ms nhờ kỹ thuật này — con số tham chiếu thực tế cho hướng tối ưu tiếp theo.

6.3.2. Bảo mật cấp cao (trung hạn)

4. **Montgomery Ladder:** Thay thế Double-and-Add bằng Montgomery Ladder để đạt *constant-time* thực sự — thuật toán luôn thực hiện đúng 1024 phép `double` và 1024 phép `add` bất kể giá trị scalar, loại bỏ hoàn toàn timing side-channel.
5. **Phép ghép cặp đầy đủ (Pairing):** Cài đặt vòng lặp Miller và bước lũy thừa cuối (final exponentiation) trên $\mathbb{F}_{p^{18}}$ để kích hoạt BLS aggregate signatures. Đây là hướng có giá trị ứng dụng cao nhất — một chữ ký BLS tổng hợp từ n người ký có thể xác minh bằng đúng 2 phép ghép cặp, không phụ thuộc vào n .
6. **Multi-Scalar Multiplication (MSM — Pippenger):** Tối ưu bước xác minh $[s]G + [e]Q$ bằng thuật toán Pippenger [20], giảm ~50% công việc so với hai phép nhân vô hướng tuần tự.

6.3.3. Mở rộng ứng dụng (dài hạn)

7. **Giao thức ngưỡng (Threshold Signatures):** Dựa trên tính tổng hợp tự nhiên của Schnorr, cài đặt các giao thức MuSig2 hay FROST cho phép t -trong- n người ký tham gia, phù hợp với ứng dụng đa chữ ký (multi-sig) trong blockchain và hệ thống tài chính.
8. **Kiểm định hình thức (Formal Verification):** Sử dụng framework như `creusot` hoặc `kani` để chứng minh tính đúng đắn của các tính chất quan trọng (ví dụ: `add` thỏa mãn luật giao hoán và kết hợp) ở mức ngôn ngữ hình thức, nâng mức độ đảm bảo an toàn lên chuẩn mực cao nhất.

Các hướng **1** và **4** là ưu tiên cao nhất vì chúng đồng thời cải thiện cả hiệu năng lẫn bảo mật mà không đòi hỏi thay đổi kiến trúc cấp cao, và có thể được triển khai độc lập trong phạm vi codebase hiện tại.

TÀI LIỆU THAM KHẢO

- [1] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976. doi: 10.1109/TIT.1976.1055638.
- [2] Michael Rosing. *Elliptic Curve Cryptography for Developers*. Manning Publications, 2024. ISBN 9781633437845.
- [3] Clifford Cocks and Robert G. E. Pinch. Identity-based cryptosystems based on the weil pairing. 2001. Unpublished manuscript.
- [4] Razvan Barbulescu and Sylvain Duquesne. Updating key size estimations for pairings. *Journal of Cryptology*, 32:1298–1336, 2019. doi: 10.1007/s00145-018-9280-5.
- [5] Elaine Barker. Recommendation for key management, part 1: General. Technical Report NIST Special Publication 800-57 Part 1 Revision 5, National Institute of Standards and Technology, 2020.
- [6] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of Computation*, 48(177):203–209, 1987.
- [7] Victor S. Miller. Use of elliptic curves in cryptography. *Advances in Cryptology – CRYPTO ’85*, pages 417–426, 1986.
- [8] Dan Shumow and Niels Ferguson. On the possibility of a back door in the nist sp800-90 dual ec prng. In *CRYPTO 2007 Rump Session*, 2007.
- [9] John M. Pollard. Monte carlo methods for index computation $(\bmod p)$. *Mathematics of Computation*, 32(143):918–924, 1978.
- [10] CVE. Cve-2014-0160: The heartbleed bug. <https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2014-0160>, 2014.
- [11] Matt Miller. Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape. BlueHat IL, 2019. <https://github.com>.

com/Microsoft/MSRC-Security-Research/blob/master/presentations/2019_02_BlueHatIL/Trends,%20challenge,%20and%20shifts%20in%20vulnerability%20mitigation.pdf.

- [12] NIST. Digital signature standard (dss). Technical Report FIPS PUB 186-4, National Institute of Standards and Technology, 2013.
- [13] C. P. Schnorr. Efficient identification and signatures for smart cards. In *Advances in Cryptology – CRYPTO ’89*, pages 239–252. Springer, 1990. doi: 10.1007/0-387-34805-0_22.
- [14] Joseph H. Silverman. *The Arithmetic of Elliptic Curves*. Springer, 2nd edition, 2009. doi: 10.1007/978-0-387-09494-6.
- [15] Lawrence C. Washington. *Elliptic Curves: Number Theory and Cryptography*. Chapman and Hall/CRC, 2nd edition, 2008. ISBN 9781420071467.
- [16] Tran Duy. curve1024: Thư viện mật mã đường cong elliptic 1024-bit viết bằng rust. <https://github.com/Tranduy1dol/curve1024>, 2026. Mã nguồn mở, MIT License.
- [17] Peter L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985. doi: 10.1090/S0025-5718-1985-0777282-X.
- [18] Least Authority. *MoonMath Manual to zk-SNARKs*. 2023. Open-source textbook on zero-knowledge proofs and elliptic curve arithmetic.
- [19] Shan Chen, Shaofang Hong, and Kunpeng Wang. Cocks–pinch curves of embedding degrees five to eight and optimal ate pairing computation. *Science China Mathematics*, 63:1519–1538, 2020. doi: 10.1007/s11425-018-9538-1.
- [20] Sean Rowe. Bls12-381: New zk-snark elliptic curve construction. <https://electriccoin.co/blog/new-snark-curve/>, 2017.
- [21] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *Advances in Cryptology – ASIACRYPT 2001*, pages 514–532. Springer, 2001. doi: 10.1007/3-540-45682-1_30.

- [22] Alfred J. Menezes, Tatsuaki Okamoto, and Scott A. Vanstone. Reducing elliptic curve logarithms to logarithms in a finite field. In *IEEE Transactions on Information Theory*, volume 39, pages 1639–1646, 1993. doi: 10.1109/18.259647.
- [23] Nigel P. Smart. The discrete logarithm problem on elliptic curves of trace one. *Journal of Cryptology*, 12(3):193–196, 1999. doi: 10.1007/s001459900052.